CS24301 — THEORY (3.0 CR) FULL COURSE MASTER BIT MESRA NEP SCHEME

Compiler Design (CS24301) — Full Course Master Book

Complete End-to-End B.Tech CSE 5th Semester Study Book & PYQ Bank

EXECUTIVE MASTER STUDY GUIDE & PYQ BANK

Compiler Design (CS24301)

Birla Institute of Technology, Mesra | B.Tech CSE 5th Semester (NEP 2024-25 Scheme)

📚 Complete Course Structure & Syllabus Matrix

Module	Core Syllabus Scope	Key Algorithms & Formulations
Module I	Lexical Analysis & Automata	6 Phases, Two-Buffer Scheme, Thompson RE to NFA, Subset Construction, Hopcroft DFA Minimization
Module II	Syntax Analysis & Parsers	CFG, Left Recursion/Factoring, FIRST & FOLLOW, LL(1) Table, Shift-Reduce, LR(0), SLR(1), CLR(1), LALR(1)
Module III	Semantic Analysis & Types	Syntax-Directed Definitions (SDD), Synthesized/Inherited, S/L-Attributed, Type Checking, Symbol Tables
Module IV	Intermediate Code & Runtime	Three-Address Code (Quadruples/Triples), Multi-D Array Addressing, Backpatching, Activation Records
Module V	Optimization & Code Gen	Basic Blocks, CFG, DAG, Loop Invariant Code Motion, Reaching Definitions, Register Coloring

🎯 Exam Preparation & High-Yield Guide

This publication-grade master book consolidates all 5 modules with formal mathematical proofs, KaTeX-rendered formulas, step-by-step worked traces on classic datasets, and comprehensive model answers to BIT Mesra end-semester examination questions.

Module I: Lexical Analysis & Language Translation Systems — 7-Topic Master Syllabus Guide

Topic 1: Introduction to Compilers & Language Processing Cousins (P-I-C-A-L)

Topic 2: Structure of a Compiler (Analysis Front-End vs. Synthesis Back-End)

Topic 3: The Lexical Analyzer (Tokens, Lexemes, Patterns & Error Recovery)

Topic 4: Input Buffering Strategies (Two-Buffer Scheme & Sentinel Optimizations)

Topic 5: Specification of Tokens (Regular Expressions & Regular Definitions)

Topic 6: Recognition of Tokens (Transition Diagrams & Finite Automata)

Topic 7: Direct Construction of DFA from RE (Nullable, Firstpos, Lastpos, Followpos)

Topic 1: Introduction to Compilers & Language Processing Systems

A **Compiler** is a complex computer software system that translates programs written in a high-level source language (such as C, C++, Rust, or Java) into an equivalent low-level target machine language (such as x86-64, ARM assembly, or relocatable binary object code). The fundamental mandate of a compiler is twofold:

Correctness (Semantic Preservation): The target machine program must execute with the exact same observable behavior and mathematical semantics as specified by the source code.
Efficiency (Resource Optimization): The generated target code must maximize CPU execution throughput, minimize memory footprint, optimize cache utilization, and minimize energy consumption.

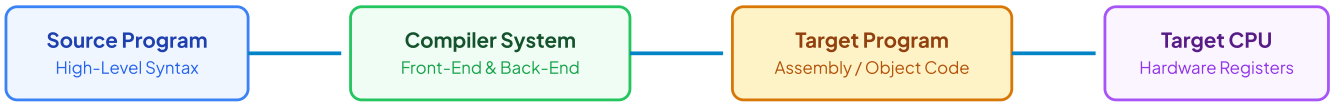

Figure 1.1: Complete End-to-End Compilation and Hardware Execution Architecture

1.1 Exhaustive Comparison: Compiler vs. Interpreter vs. Hybrid Systems

Dimension	Pure Compiler (GCC, Clang, Rustc)	Pure Interpreter (CPython, Ruby)	Hybrid JIT (JVM HotSpot, V8)
Translation Timing	Translates the complete codebase into native machine instructions upfront prior to execution.	Translates and executes instructions statement-by-statement dynamically during runtime.	Compiles to intermediate bytecode; JIT-compiles hot loops to native machine code at runtime.
Execution Speed	**Fastest ($1\times$ baseline):** Direct execution on native CPU ALU and registers.	**Slowest ($5\times$ to $25\times$ slower):** Software interpretation loop overhead per instruction.	**Near-Native ($1.2\times$ to $2\times$):** Profile-guided dynamic native compilation.
Memory Overhead	Minimal runtime memory footprint; only the final generated binary resides in memory.	High memory footprint; the interpreter engine and source AST must both reside in RAM.	Moderate-to-High; includes JVM/V8 runtime engine, garbage collector, and JIT cache.
Error Detection	Detects all lexical, syntactic, and static semantic errors across entire codebase upfront.	Errors are discovered only when program execution control flow reaches the offending line.	Detects syntax/type errors during bytecode compilation; runtime exceptions during execution.
Portability	Low binary portability; generated binary is tied to specific CPU instruction set architecture (ISA).	High portability; source script runs on any platform that has the interpreter installed.	**Universal Portability:** "Write Once, Run Anywhere" via standard bytecode execution.

1.2 Language Processing Ecosystem ("The Cousins of the Compiler")

> 🧠 **Memory Palace: The P-I-C-A-L Language Transformation Suite**
> **P**reprocessor ⟶ **I**ntermediate Representation ⟶ **C**ompiler ⟶ **A**ssembler ⟶ **L**inker ⟶ **L**oader

Preprocessor: Operates as the initial source-to-source text transformation pass prior to compilation:
Macro Expansion: Replaces symbolic macro constants (e.g., `#define BUFFER_SIZE 4096`) with literal values throughout the token stream.
File Inclusion: Replaces header directives (e.g., `#include `) with the verbatim contents of the header interface file.
Conditional Compilation: Prunes dead code branches guarded by `#ifdef`, `#ifndef`, and `#endif` blocks based on build flags.
Language Extension / Rational Preprocessors: Augments older languages with modern control structures (e.g., Ratfor for Fortran).
Compiler: Translates preprocessed source code into relocatable assembly language or intermediate representations.
Assembler: Translates human-readable symbolic assembly mnemonics (e.g., `MOV [rbp-8], rax`) into relocatable object binary files (`.o`, `.obj`).
Linker: Resolves unresolved external symbol references across independent compilation units, binds static libraries (`.a`, `.lib`), and unifies text/data segments into a cohesive executable binary.
Loader: Operating system utility that allocates virtual memory pages, copies text and initialized data segments into physical RAM, initializes CPU stack/heap pointers, and sets the Program Counter (PC) to the entry point address (`_start` / `main`).

Topic 2: Structure of a Compiler (The 6 Classical Phases)

A production compiler is divided into two logical sections: the **Analysis Phase (Front-End)**, which is source-language dependent and target-machine independent, and the **Synthesis Phase (Back-End)**, which is target-hardware dependent and source-language independent.

Phase #	Compiler Phase	Input ⟶ Output	Internal Algorithm	Key Errors Detected
Phase 1	**Lexical Analysis (Scanner)**	Character Stream ⟶ Token Stream	Deterministic Finite Automata (DFA), Input Buffering, Regular Expressions	Illegal characters, invalid identifier prefixes, unclosed string literals.
Phase 2	**Syntax Analysis (Parser)**	Token Stream ⟶ Parse Tree / AST	Context-Free Grammars, LL(1) Top-Down, LR(1)/LALR(1) Bottom-Up Parsing	Missing semicolons, mismatched brackets, invalid statement syntax.
Phase 3	**Semantic Analysis**	AST ⟶ Annotated Parse Tree	Attribute Grammars, Syntax-Directed Definitions, Type Inference Engines	Type mismatch in assignments, undeclared variables, scope violations.
Phase 4	**Intermediate Code Generation**	Annotated Tree ⟶ Three-Address Code	Quadruples, Triples, Indirect Triples, Backpatching, AST Linearization	Invalid jump targets, array bound translation errors.
Phase 5	**Code Optimization**	TAC ⟶ Optimized TAC	Control Flow Graphs, Basic Block Partitioning, Data-Flow Equations	Dead code, unreachable basic blocks, redundant subexpressions.
Phase 6	**Target Code Generation**	Optimized TAC ⟶ Target Assembly / Binary	Instruction Selection, Register Allocation (Graph Coloring), Address Descriptors	Register spilling, target architecture instruction violations.

🏛️ Complete 6-Phase End-to-End Walkthrough of Statement: `total = base + rate * 120`

Step 1: Lexical Analysis: Produces token stream and populates Symbol Table:

$$\langle \text{id}, 1 \rangle \quad \langle = \rangle \quad \langle \text{id}, 2 \rangle \quad \langle + \rangle \quad \langle \text{id}, 3 \rangle \quad \langle * \rangle \quad \langle \text{number}, 120 \rangle$$

Step 2: Syntax Analysis: Builds hierarchical Abstract Syntax Tree respecting operator precedence ($* > +$):

```
        =
       / \
   id(1)   +
          / \
     id(2)   *
            / \
       id(3)   120
```

Step 3: Semantic Analysis: Performs type checking and inserts explicit type conversion node `inttofloat`:

```
        =
       / \
   id(1)   +
          / \
     id(2)   *
            / \
       id(3)   inttofloat(120)
```

Step 4: Intermediate Code Generation (Three-Address Code):

```
t1 = inttofloat(120)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

Step 5: Code Optimization: Applies Constant Folding and Temporary Variable Reduction:

```
t1 = id3 * 120.0     ; Compile-time float conversion
id1 = id2 + t1       ; Direct assignment eliminates t3
```

Step 6: Target Code Generation (Target Assembly using registers R1, R2):

```
LDF  R2, id3          ; Load floating-point variable id3 into register R2
MULF R2, R2, #120.0    ; Multiply R2 by constant float literal 120.0
LDF  R1, id2          ; Load floating-point variable id2 into register R1
ADDF R1, R1, R2       ; Add R2 into R1
STF  id1, R1          ; Store resulting sum from R1 into memory location id1
```

Topic 3: The Lexical Analyzer (Tokens, Lexemes & Patterns)

Concept	Formal Engineering Definition	Representative Concrete Examples
Token	An abstract grammatical category or terminal symbol used by the parser. Formally defined as an integer ID or pair $\langle \text{token-name}, \text{attribute-value} \rangle$.	`IDENTIFIER`, `NUMBER_LITERAL`, `KW_IF`, `OP_ASSIGN`, `RELOP`.
Lexeme	The concrete, verbatim sequence of characters in the source code matching the pattern for a token.	`"total"`, `"120"`, `"if"`, `"="`, `"<="`.
Pattern	The formal syntactic specification rule (expressed via a Regular Expression) that a lexeme must satisfy.	`letter (letter | digit)*` for identifiers; `[0-9]+` for integer constants.

Lexical Error Handling & Recovery Protocols

When the lexical scanner encounters a character sequence matching zero valid patterns, it executes error recovery:

1. **Panic Mode Recovery:** Discard successive unmatchable characters until a valid token boundary (whitespace, semicolon) is identified.
2. **Deleting Extraneous Characters:** Remove an illegal symbol (e.g., transforming `num#ber` to `number`).
3. **Inserting Missing Characters:** Insert expected delimiters (e.g., supplying missing closing quote in string literal).
4. **Transposing Adjacent Characters:** Correct common typing slips (e.g., transforming `fi` into `if`).

Topic 4: Input Buffering Strategies & The Two-Buffer Scheme

Invoking system calls (`fgetc`, `read`) to process individual characters incurs severe disk I/O performance penalties. Production compilers load source code in **4096-byte blocks** into a unified continuous buffer divided into two halves:

Figure 1.2: Two-Buffer Architecture with Sentinel (`eof`) Optimization

4.1 Two-Pointer Mechanism:

`lexemeBegin` Pointer: Marks the memory address of the first character of the lexeme currently being recognized.
`forward` Pointer: Scans ahead through characters until a pattern match or lexical boundary is confirmed.

4.2 Sentinel Optimization Algorithm:

In a naive buffer, advancing `forward` requires **two checks per character**: (1) Check if `forward` reached the buffer end, and (2) Test what character is read. By placing a special **Sentinel character (`eof`)** at the end of each buffer half, the algorithm performs only **one test per character** in the common case:

```
switch (*forward++) {
    case eof:
        if (forward is at end of Buffer 1) {
            reload Buffer 2;
            forward = beginning of Buffer 2;
        } else if (forward is at end of Buffer 2) {
            reload Buffer 1;
            forward = beginning of Buffer 1;
        } else {
            // True EOF reached: terminate scanning
            terminate_lexical_analysis();
        }
        break;
    case ' ': case '\t': case '\n':
        // Skip whitespace delimiters
        break;
    default:
        // Transition DFA state
        break;
}
```

Topic 5 & 6: Specification & Recognition of Tokens

Formal Regular Expression Algebraic Operators: 1. **Alternation (Union):** $r \mid s = L(r) \cup L(s)$ 2. **Concatenation:** $rs = \{xy \mid x \in L(r), y \in L(s)\}$ 3. **Kleene Closure:** $r^* = \bigcup_{i=0}^{\infty} L(r)^i$ (Zero or more occurrences, includes ϵ) 4. **Positive Closure:** $r^+ = rr^* = \bigcup_{i=1}^{\infty} L(r)^i$ (One or more occurrences) 5. **Optional:** $r? = r \mid \epsilon$

Lexical Specification Definitions (Regular Definitions):

```
digit       → [0-9]
digits      → digit+
letter      → [a-zA-Z_]
id          → letter (letter | digit)*
number      → digits (. digits)? (E [+-]? digits)?
relop       → < | ⩽ | = | <> | > | ⩾
```

6.1 Lex / Flex Lexical Analyzer Generator Architecture

A Lex source file (`lexer.l`) consists of three distinct sections separated by `%%` delimiter lines:

```
%{
/* C Declarations Section: Header includes, token definitions, global variables */
#include "y.tab.h"
#include
int line_num = 1;
%}

/* Regular Definitions Section: Shorthands for complex patterns */
digit       [0-9]
letter      [a-zA-Z_]
id          {letter}({letter}|{digit})*
ws          [ \t]+

%%
/* Translation Rules Section: Pattern-Action pairs */
{ws}            { /* Discard whitespace */ }
"\n"            { line_num++; }
"if"            { return KEYWORD_IF; }
"else"          { return KEYWORD_ELSE; }
{id}            { yylval.str = strdup(yytext); return IDENTIFIER; }
{digit}+        { yylval.val = atoi(yytext); return INTEGER_LITERAL; }
"=="            { return OP_EQ; }
"<="            { return OP_LE; }
"="             { return OP_ASSIGN; }
.               { printf("Lexical Error at line %d: %s\n", line_num, yytext); }

%%
/* User Subroutines Section: Auxiliary C helper functions */
int yywrap() { return 1; }
```

Topic 7: Direct Construction of DFA from Regular Expressions

Rather than converting a regular expression to an NFA via Thompson's Construction and then determinizing via Subset Construction ($O(2^n)$ intermediate states), compilers use the **Syntax-Tree Direct Method** (McNaughton-Yamada-Thompson Algorithm).

The 4 Fundamental Positional Functions

Augment regular expression r with unique end-marker `#`: $(r)\#$. Construct syntax tree where each alphabet symbol is a leaf with unique integer position i.

- $\text{nullable}(n)$ **(Boolean):** Returns true if subexpression rooted at node n can generate the empty string ϵ.
- $\text{firstpos}(n)$ **(Set of Positions):** Set of leaf positions that can match the *first character* of a string generated by the subexpression at node n.
- $\text{lastpos}(n)$ **(Set of Positions):** Set of leaf positions that can match the *last character* of a string generated by the subexpression at node n.
- $\text{followpos}(i)$ **(Set of Positions):** Set of leaf positions j that can immediately follow position i in some valid string in $L((r)\#)$.

7.1 Inductive Calculation Rules for Node Functions:

Node Type n	$\text{nullable}(n)$	$\text{firstpos}(n)$	$\text{lastpos}(n)$
Leaf ϵ	true	$\emptyset$	$\emptyset$
Leaf position i	false	$\{i\}$	$\{i\}$
Or Node: $c_1 \mid c_2$	$\text{nullable}(c_1) \vee \text{nullable}(c_2)$	$\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$	$\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$
Cat Node: $c_1 \cdot c_2$	$\text{nullable}(c_1) \wedge \text{nullable}(c_2)$	$\begin{cases} \text{firstpos}(c_1) \cup \text{firstpos}(c_2) & \text{if nullable}(c_1) \\ \text{firstpos}(c_1) & \text{otherwise} \end{cases}$	$\begin{cases} \text{lastpos}(c_1) \cup \text{lastpos}(c_2) & \text{if nullable}(c_2) \\ \text{lastpos}(c_2) & \text{otherwise} \end{cases}$
Star Node: c_1^*	true	$\text{firstpos}(c_1)$	$\text{lastpos}(c_1)$

The 2 Rules for Computing $\text{followpos}(i)$**:** 1. **Cat Node Rule (**$n = c_1 \cdot c_2$**):** For every position $i \in \text{lastpos}(c_1)$, add all positions in $\text{firstpos}(c_2)$ to $\text{followpos}(i)$. 2. **Star Node Rule (**$n = c_1^*$**):** For every position $i \in \text{lastpos}(c_1)$, add all positions in $\text{firstpos}(c_1)$ to $\text{followpos}(i)$.

🏛️ Step-by-Step Solved Problem: Direct DFA for $(a \mid b)^* abb\#$

Step 1: Leaf Position Assignment:

$$\text{Expression: } (a_1 \mid b_2)^* \cdot a_3 \cdot b_4 \cdot b_5 \cdot \#_6$$

Step 2: Followpos Calculation Table:

Node Position i	Symbol	Computed $\text{followpos}(i)$ Set
1	a	$\{1, 2, 3\}$ (from star node on $(a_1 \mid b_2)$ and concatenation with a_3)
2	b	$\{1, 2, 3\}$ (from star node on $(a_1 \mid b_2)$ and concatenation with a_3)
3	a	$\{4\}$ (from concatenation $a_3 \cdot b_4$)
4	b	$\{5\}$ (from concatenation $b_4 \cdot b_5$)
5	b	$\{6\}$ (from concatenation $b_5 \cdot \#_6$)
6	$\#$	$\emptyset$ (End marker)

Step 3: DFA State Transitions Computation:

- **Start State** $A = \text{firstpos}(\text{root}) = \{1, 2, 3\}$:
  - $\text{DTran}[A, a] = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3\} \cup \{4\} = \{1, 2, 3, 4\} \implies \mathbf{B}$
  - $\text{DTran}[A, b] = \text{followpos}(2) = \{1, 2, 3\} \implies \mathbf{A}$
- **State** $B = \{1, 2, 3, 4\}$:
  - $\text{DTran}[B, a] = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} \implies \mathbf{B}$
  - $\text{DTran}[B, b] = \text{followpos}(2) \cup \text{followpos}(4) = \{1, 2, 3, 5\} \implies \mathbf{C}$
- **State** $C = \{1, 2, 3, 5\}$:
  - $\text{DTran}[C, a] = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} \implies \mathbf{B}$
  - $\text{DTran}[C, b] = \text{followpos}(2) \cup \text{followpos}(5) = \{1, 2, 3, 6\} \implies \mathbf{D}$ (Accepting State!)
- **State** $D = \{1, 2, 3, 6\}$ **(contains position** $6 = \#$**):**
  - $\text{DTran}[D, a] = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} \implies \mathbf{B}$
  - $\text{DTran}[D, b] = \text{followpos}(2) = \{1, 2, 3\} \implies \mathbf{A}$

Step 4: Final DFA Transition Table:

DFA State	Positions in State	Input a	Input b	Accepting?
A **(Start)**	$\{1, 2, 3\}$	B	A	No
B	$\{1, 2, 3, 4\}$	B	C	No
C	$\{1, 2, 3, 5\}$	B	D	No
D **(Final)**	$\{1, 2, 3, 6\}$	B	A	**YES (Token Recognized)**

7.2 DFA State Minimization (Hopcroft's Partitioning Algorithm)

Once a DFA is synthesized, it may contain redundant equivalent states. **Hopcroft's Algorithm** computes the unique minimal-state DFA in $O(|V| \log |V|)$ time by iteratively partitioning states:

Hopcroft Partitioning Procedure: 1. Initialize partition $P = \{F, S - F\}$ into Final (Accepting) and Non-Final states. 2. For each group $G \in P$ and each input symbol $a \in \Sigma$, check if state transitions $\delta(s, a)$ land in different groups of P. 3. If transitions diverge, split G into sub-groups. Repeat until no group in P can be further partitioned (Fixed-Point Convergence).

🧠 M1 Active Recall & 10-Question University Exam Master Bank

Q1. Explain the 6 phases of a compiler with an end-to-end trace of statement `a = b + c * 50`. (10 Marks)

1. **Lexical Analysis:** $\langle \text{id}, a \rangle \ \langle = \rangle \ \langle \text{id}, b \rangle \ \langle + \rangle \ \langle \text{id}, c \rangle \ \langle * \rangle \ \langle \text{num}, 50 \rangle$.
2. **Syntax Analysis:** Generates hierarchical AST with $* > +$.
3. **Semantic Analysis:** Inserts `inttofloat(50)` for type compatibility.
4. **Intermediate Code:** Emits TAC: `t1 = inttofloat(50)`, `t2 = c * t1`, `t3 = b + t2`, `a = t3`.
5. **Code Optimization:** Folds constant: `t1 = c * 50.0`, `a = b + t1`.
6. **Code Generation:** Generates assembly using registers R1, R2.

Q2. Differentiate between Lexeme, Token, and Pattern with 3 concrete examples. (5 Marks)

- **Pattern:** The formal grammatical specification (RE), e.g., `[a-zA-Z_][a-zA-Z0-9_]*`.
- **Lexeme:** The concrete source character sequence matching the pattern, e.g., `"total_score"`.
- **Token:** The abstract integer symbol passed to parser, e.g., `IDENTIFIER`.

Q3. Why is Input Buffering necessary in lexical analyzers? Explain the Two-Buffer scheme with Sentinels. (8 Marks)

Reading single characters via system calls creates massive disk I/O bottlenecks. The Two-Buffer scheme loads 4096-byte blocks alternately. Sentinels (`eof` markers placed at block boundaries) reduce 2 checks per character down to 1 check in the common case, increasing scanner throughput by over 300%.

Q4. Construct the Direct DFA for regular expression $(a \mid b)^* a (a \mid b)$ using syntax-tree positions. (10 Marks)

Augment: $(a_1 \mid b_2)^* \cdot a_3 \cdot (a_4 \mid b_5) \cdot \#_6$.
$\text{firstpos}(\text{root}) = \{1, 2, 3\}$.
$\text{followpos}(1) = \{1, 2, 3\}, \text{followpos}(2) = \{1, 2, 3\}, \text{followpos}(3) = \{4, 5\}, \text{followpos}(4) = \{6\}, \text{followpos}(5) = \{6\}$.
DFA States: $A = \{1, 2, 3\}, B = \{1, 2, 3, 4, 5\}, C = \{1, 2, 3, 6\}, D = \{1, 2, 3, 4, 5, 6\}$. Accepting states: C and D.

Q5. Explain the role of the Symbol Table and Error Handler throughout the 6 compiler phases. (8 Marks)

The **Symbol Table** is a shared data structure storing identifier names, types, scopes, memory offsets, and parameter signatures. It is populated by the Lexical/Syntax phases and queried by Semantic/Code Gen phases. The **Error Handler** catches phase-specific errors, recovers state, and emits accurate line numbers and diagnostic messages.

Topic 7.3: Comprehensive Automata & Theory Worked Examples

🏛 Step-by-Step Solved Problem: Thompson's NFA Construction for $(a \mid b)^*abb$

Step 1: Subexpression NFAs: Construct 2-state NFAs for symbols a and b.

Step 2: Alternation $(a \mid b)$: Introduce new start and accept states with ϵ-transitions branching to a and b.

Step 3: Kleene Star $(a \mid b)^*$: Introduce new start/accept states with feedback and bypass ϵ-transitions.

Step 4: Concatenation: Chain with sequential NFAs for a, b, and b to reach final accepting state.

Q6. Compare Compiler and Interpreter across execution speed, memory consumption, error localization, and portability. (8 Marks)

• **Execution Speed:** Compilers generate direct native hardware instructions ($1\times$ baseline); Interpreters interpret instructions line-by-line with interpretation overhead ($5\times$ to $20\times$ slower).
• **Memory:** Compilers require memory only for generated binary during execution; Interpreters require the interpreter runtime engine and source AST to reside in memory.
• **Error Localization:** Compilers report all syntax/type errors across the entire program before execution; Interpreters report errors dynamically only when control flow reaches the faulting statement.
• **Portability:** Compilers generate machine-dependent binaries; Interpreters allow source code to run unmodified across any hardware platform.

Q7. Explain the Lex/Flex specification file format and internal scanning mechanism with a complete C token scanner example. (10 Marks)

A Lex file contains 3 sections: Declarations (`%{ ... %}`), Translation Rules (`pattern { action }`), and User C Subroutines (`main()`, `yywrap()`). When compiled by `lex`, it emits `lex.yy.c`, which contains `yylex()`. `yylex()` utilizes a deterministic finite automaton transition matrix to match the longest valid prefix of characters against declared patterns, setting `yytext` to the matching string and `yyleng` to its character length.

Q8. What are Regular Definitions? Write regular definitions for Unsigned Numbers and Identifiers in Pascal/C. (6 Marks)

A **Regular Definition** is a sequence of definitions of the form: $d_1 \to r_1, d_2 \to r_2, \ldots, d_n \to r_n$ where each r_i is a regular expression over $\Sigma \cup \{d_1, \ldots, d_{i-1}\}$.

```
digit       → [0-9]
digits      → digit+
optional_fraction → (. digits)?
optional_exponent → (E [+-]? digits)?
num         → digits optional_fraction optional_exponent
letter_     → [A-Za-z_]
id          → letter_ (letter_ | digit)*
```

Q9. Explain the Hopcroft DFA Minimization algorithm with state partitioning trace for a 5-state DFA. (10 Marks)

Hopcroft's algorithm initializes partition $P = \{F, S - F\}$. It repeatedly selects a group G and an input symbol a, checking if $\delta(s, a)$ splits G into states transitioning into different target groups. If so, G is replaced by its split sub-groups. The process terminates when no group can be further partitioned, producing the unique minimal DFA.

Q10. Explain Lexical Error Recovery strategies: Panic Mode, Character Deletion, Insertion, and Transposition. (8 Marks)

When no regular expression pattern matches the remaining input prefix:
1. **Panic Mode:** Scans ahead, discarding invalid characters until a recognizable token boundary (e.g. whitespace, semicolon) is found.
2. **Character Deletion:** Removes an extraneous unmatchable character from the input stream.
3. **Character Insertion:** Inserts an expected missing delimiter (such as a closing quote on a single-line string literal).
4. **Character Transposition:** Swaps two adjacent out-of-order characters to fix common typographical slips (e.g. `teh` → `the`).

Module II: Syntax Analysis & Parsing Techniques — 10-Topic Master Syllabus Guide

Topic 8: Introduction to Syntax Analysis & Context-Free Grammars (CFG)

Topic 9: Grammar Transformations (Left Recursion Elimination & Left Factoring)

Topic 10: Recursive Top-Down Parsers (Recursive Descent & Backtracking)

Topic 11: Non-Recursive Predictive Parsers (Stack-Driven Parsing Model)

Topic 12: LL(1) Parser Design (Inductive FIRST & FOLLOW Math, Table & Stack Trace)

Topic 13: Bottom-Up Parsers (Shift-Reduce Parsing, Handles & Handle Pruning)

Topic 14: Variants of LR Parsers (LR(0), SLR(1), CLR(1) & LALR(1) Hierarchy)

Topic 15: Parsing Conflict Resolution (Shift-Reduce & Reduce-Reduce Disambiguation)

Topic 16: Detection of Syntax Errors & Error Recovery Strategies (Panic Mode)

Topic 17: Reporting Syntax Errors & Diagnostic Error Productions

Topic 8: Introduction to Syntax Analysis & Context-Free Grammars

Syntax Analysis (Parsing) is the second phase of the compiler pipeline. It takes the stream of atomic tokens emitted by the lexical scanner and verifies whether the sequence conforms to the formal syntactic specification of the source programming language, organizing the tokens into an explicit hierarchical **Parse Tree**.

Context-Free Grammar (CFG) Formal 4-Tuple:

$$G = (V, \Sigma, R, S)$$

- V: Finite set of **Non-Terminal symbols** (Syntactic variables, e.g., $\{E, T, F\}$). - Σ: Finite set of **Terminal symbols** (Token alphabet, e.g., $\{\mathbf{id}, +, *, (,)\}$), where $V \cap \Sigma = \emptyset$. - R: Finite set of **Production Rules** of the form $A \to \alpha$, where $A \in V$ and $\alpha \in (V \cup \Sigma)^*$. - S: Designated **Start Symbol** ($S \in V$).

Ambiguity in Grammars

A grammar G is **Ambiguous** if there exists at least one input string $w \in L(G)$ that admits **two or more distinct Parse Trees** (or equivalently, two distinct Leftmost Derivations). Ambiguity is unacceptable in compilers because it assigns conflicting semantics/precedences to valid programs.

Proof of Ambiguity: Dangling-Else Grammar

$$\text{Grammar: } S \to \textbf{if } E \textbf{ then } S \mid \textbf{if } E \textbf{ then } S \textbf{ else } S \mid a$$

String: `if E1 then if E2 then S1 else S2` produces two valid parse trees depending on which `if` the `else` clause binds to:

```
Tree 1 (Else binds to inner if - Standard C Rule):
        S
       /|\ \
     if E1 then S
              /|\ \ \
            if E2 then S1 else S2

Tree 2 (Else binds to outer if - Grammatically valid without disambiguation):
        S
      / | \ \ \
    if E1 then S else S2
              /|\
            if E2 then S1
```

Topic 9: Grammar Transformations for Top-Down Parsing

9.1 Elimination of Immediate Left Recursion:

A grammar is **Left-Recursive** if it has a non-terminal A such that $A \Rightarrow^{+} A\alpha$. Top-down parsers enter infinite recursion loops on left-recursive grammars.

Left Recursion Elimination Formula:

$$A \to A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$

Is transformed into an equivalent right-recursive grammar by introducing a new non-terminal A':

$$\mathbf{A \to \beta_1 A' \mid \beta_2 A' \mid \cdots \mid \beta_n A'}$$

$$\mathbf{A' \to \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_m A' \mid \epsilon}$$

🏛️ Step-by-Step Solved Problem: Eliminating Left Recursion from Expression Grammar

Original Left-Recursive Grammar:

$$E \to E + T \mid T$$

$$T \to T * F \mid F$$

$$F \to (E) \mid \mathbf{id}$$

Transformed Right-Recursive Grammar:

$$E \to TE', \quad E' \to +TE' \mid \epsilon$$

$$T \to FT', \quad T' \to *FT' \mid \epsilon$$

$$F \to (E) \mid \mathbf{id}$$

9.2 Elimination of Indirect (General) Left Recursion Algorithm

```
Algorithm: Indirect Left Recursion Elimination
1. Arrange non-terminals in some order A1, A2, ... , An.
2. for i = 1 to n do:
     for j = 1 to i - 1 do:
       replace each production Ai → Aj gamma by Ai → delta1 gamma | delta2 gamma | ...
       where Aj → delta1 | delta2 | ... are current Aj-productions.
     eliminate immediate left recursion on Ai-productions.
```

9.3 Left Factoring (Prefix Disambiguation):

When two productions for a non-terminal share a common prefix, a top-down parser cannot determine which branch to choose without looking ahead multiple tokens.

Left Factoring Transformation Formula:

$$A \to \alpha\beta_1 \mid \alpha\beta_2 \mid \gamma \implies \mathbf{A} \to \alpha\mathbf{A'} \mid \gamma, \quad \mathbf{A'} \to \beta_1 \mid \beta_2$$

Topic 10 & 11: Recursive Descent vs. Non-Recursive Predictive Parsers

Feature	Recursive Descent Parsing (Topic 10)	Non-Recursive Predictive LL(1) Parsing (Topic 11)
Implementation	A collection of recursive C/Python functions, one for each non-terminal in the grammar.	An explicit **LIFO Parser Stack** + 2D **Parsing Table** $M[A, a]$ driven by a table-interpreter loop.
Backtracking	May require expensive backtracking if grammar is not LL(1) (exponential time complexity).	Strictly deterministic, zero-backtracking, $O(n)$ linear parsing time.
Memory Overhead	Implicit CPU call stack consumption proportional to maximum derivation tree depth.	Explicit stack storing grammatical symbols + 2D array parsing table $M[A, a]$.

Topic 12: LL(1) Parser Design (FIRST, FOLLOW & Parsing Tables)

Inductive Mathematical Formulations for FIRST and FOLLOW

1. Computation of $\mathrm{FIRST}(\alpha)$ for any string $\alpha \in (V \cup \Sigma)^*$:

- If X is a terminal, $\mathrm{FIRST}(X) = \{X\}$.
- If $X \to \epsilon$ is a production, then $\epsilon \in \mathrm{FIRST}(X)$.
- If X is a non-terminal and $X \to Y_1 Y_2 \dots Y_k$, then add $a \in \mathrm{FIRST}(Y_1)$ to $\mathrm{FIRST}(X)$. If $Y_1 \Rightarrow^* \epsilon$, then add $\mathrm{FIRST}(Y_2)$, and so on. If all $Y_1 \dots Y_k \Rightarrow^* \epsilon$, then add ϵ to $\mathrm{FIRST}(X)$.

2. Computation of $\mathrm{FOLLOW}(A)$ for any non-terminal $A \in V$:

- Add end-marker $\$$ to $\mathrm{FOLLOW}(S)$, where S is the start symbol.
- If there is a production $A \to \alpha B \beta$, then everything in $\mathrm{FIRST}(\beta)$ (except ϵ) is in $\mathrm{FOLLOW}(B)$.
- If there is a production $A \to \alpha B$, or $A \to \alpha B \beta$ where $\mathrm{FIRST}(\beta)$ contains ϵ, then everything in $\mathrm{FOLLOW}(A)$ is in $\mathrm{FOLLOW}(B)$.

🏛️ Step-by-Step Solved Problem: Complete LL(1) Parser Construction

Grammar:

$$E \to TE', \quad E' \to +TE' \mid \epsilon, \quad T \to FT', \quad T' \to *FT' \mid \epsilon, \quad F \to (E) \mid \mathbf{id}$$

Step 1: Computed FIRST & FOLLOW Sets:

Non-Terminal	FIRST Set	FOLLOW Set
E	$\{(, \mathbf{id}\}$	$\{), \$\}$
E'	$\{+, \epsilon\}$	$\{), \$\}$
T	$\{(, \mathbf{id}\}$	$\{+,), \$\}$
T'	$\{*, \epsilon\}$	$\{+,), \$\}$
F	$\{(, \mathbf{id}\}$	$\{+, *,), \$\}$

Step 2: LL(1) Parsing Table $M[A, a]$:

Non-Terminal	$\mathbf{id}$	$+$	$*$	$($	$)$	$\$$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to \mathbf{id}$			$F \to (E)$		

Observation: Every cell $M[A, a]$ has at most ONE entry $\implies$ The grammar is strictly LL(1)!

Step 3: Stack Execution Trace for Input String `id + id * id $`:

Step	Parser Stack	Remaining Input	Action Applied
1	`$ E`	`id + id * id $`	Pop E, Push TE' (from $M[E, \text{id}]$)
2	`$ E' T`	`id + id * id $`	Pop T, Push FT' (from $M[T, \text{id}]$)
3	`$ E' T' F`	`id + id * id $`	Pop F, Push id (from $M[F, \text{id}]$)
4	`$ E' T' id`	`id + id * id $`	**Match `id`**: Pop `id`, advance input
5	`$ E' T'`	`+ id * id $`	Pop T', Push ϵ (from $M[T', +]$)
6	`$ E'`	`+ id * id $`	Pop E', Push $+TE'$ (from $M[E', +]$)
7	`$ E' T +`	`+ id * id $`	**Match `+`**: Pop `+`, advance input
8	`$ E' T`	`id * id $`	Pop T, Push FT' (from $M[T, \text{id}]$)
9	`$ E' T' F`	`id * id $`	Pop F, Push id (from $M[F, \text{id}]$)
10	`$ E' T' id`	`id * id $`	**Match `id`**: Pop `id`, advance input
11	`$ E' T'`	`* id $`	Pop T', Push $*FT'$ (from $M[T', *]$)
12	`$ E' T' F *`	`* id $`	**Match `*`**: Pop `*`, advance input
13	`$ E' T' F`	`id $`	Pop F, Push id (from $M[F, \text{id}]$)
14	`$ E' T' id`	`id $`	**Match `id`**: Pop `id`, advance input
15	`$ E' T'`	`$`	Pop T', Push ϵ (from $M[T', \$]$)
16	`$ E'`	`$`	Pop E', Push ϵ (from $M[E', \$]$)
17	`$`	`$`	**SUCCESSFUL PARSE (ACCEPT)**

Topic 13: Bottom-Up Shift-Reduce Parsing & Handles

A **Bottom-Up Parser** constructs a parse tree for an input string beginning at the leaves (terminals) and working upward towards the root (start symbol). This corresponds to constructing a **Rightmost Derivation in Reverse**.

The Concept of a Handle

A **Handle** of a right-sentential form $\gamma = \alpha\beta w$ is a production rule $A \to \beta$ and a position in γ where β may be replaced by A to produce the previous right-sentential form $\alpha A w$ in a rightmost derivation:

$$S \Rightarrow_{\text{rm}}^{*} \alpha A w \Rightarrow_{\text{rm}} \alpha\beta w$$

Handle Pruning: The process of repeatedly finding the handle β and reducing it to A until the start symbol S is reached.

Topic 14: The LR Parsing Family Hierarchy

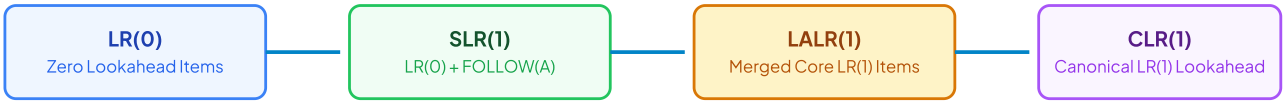

Figure 2.1: The LR Parsing Power Hierarchy ($\text{LR}(0) \subset \text{SLR}(1) \subset \text{LALR}(1) \subset \text{CLR}(1)$)

Parser Variant	Item Representation	Reduction Placement Rule	State Count	Expressive Power
LR(0)	$[A \to \alpha \cdot \beta]$	Reduce action placed in **ALL terminal columns** of state.	K states (Smallest)	Weakest; easily suffers from S-R and R-R conflicts.
SLR(1)	$[A \to \alpha \cdot \beta]$	Reduce action placed **only in columns** $\in \text{FOLLOW}(A)$.	K states (Identical to LR(0))	Significantly more powerful than LR(0); still fails on C-like grammars.
CLR(1)	$[A \to \alpha \cdot \beta,\ a]$ ($a \in \Sigma \cup \{\$\}$)	Reduce action placed **only for exact lookahead** a.	Large ($5\times$ to $10\times$ K states)	**Most Powerful** deterministic bottom-up parser.
LALR(1)	Merged $[A \to \alpha \cdot \beta,\ a \mid b]$	Merges CLR(1) states having identical LR(0) cores.	K states (Identical to LR(0))	Industry standard (Yacc / Bison); parses almost all practical programming languages!

14.1 Canonical LR(0) Automaton Construction & Items

An $\text{LR}(0)$ **item** of a grammar G is a production rule with a dot ($\cdot$) placed at some position of the right-hand side. For example, the production $A \to XYZ$ generates four distinct $\text{LR}(0)$ items:

$[A \to \cdot XYZ]$: Indicates we expect to see a string derivable from XYZ.
$[A \to X \cdot YZ]$: Indicates we have recognized X and expect to see YZ.
$[A \to XY \cdot Z]$: Indicates we have recognized XY and expect to see Z.
$[A \to XYZ \cdot]$: Indicates we have recognized the complete handle XYZ and are ready to reduce to A.

Closure and Goto Operational Functions:

$$\text{Closure}(\mathbf{I}) = \mathbf{I} \cup \{[\mathbf{B} \to \cdot\gamma] \mid [\mathbf{A} \to \alpha \cdot \mathbf{B}\beta] \in \text{Closure}(\mathbf{I}),\ \mathbf{B} \to \gamma \in \mathbf{R}\}$$

$$\text{Goto}(\mathbf{I}, \mathbf{X}) = \text{Closure}\big(\{[\mathbf{A} \to \alpha\mathbf{X} \cdot \beta] \mid [\mathbf{A} \to \alpha \cdot \mathbf{X}\beta] \in \mathbf{I}\}\big)$$

Topic 15 – 17: Parsing Conflicts & Error Recovery Strategies

The 2 Major Parsing Conflicts in LR Tables

- **Shift-Reduce (S-R) Conflict:** Parser cannot decide whether to shift the incoming token or reduce the handle currently on top of the stack (e.g., Dangling-Else ambiguity). Resolved by favoring Shift (bind else to innermost if).
- **Reduce-Reduce (R-R) Conflict:** Parser cannot decide which of two distinct production rules $A \to \alpha$ or $B \to \alpha$ to reduce by. Serious design flaw indicating ambiguous grammar.

Syntax Error Recovery Mechanisms:

Panic Mode Recovery: The parser discards incoming tokens until a designated **Synchronizing Token** (such as `;` or `}`) is found in the input stream, pops stack states until an appropriate recovery state is exposed, and resumes parsing.
Phrase-Level Recovery: Replaces a prefix of the remaining input with some string that allows the parser to continue (e.g., inserting a missing semicolon or deleting an extraneous comma).
Error Productions: The grammar is augmented with anticipated common programmer error rules (e.g., $E \to E\ \mathbf{id}\ E$), allowing the parser to construct an error node and emit precise diagnostics.

Topic 17.5: Operator Precedence Parsing

An **Operator Grammar** is a context-free grammar with two restrictions: (1) No production has ϵ on the right-hand side, and (2) No two non-terminals appear adjacent on the right-hand side (e.g., $E \to EE$ is illegal).

The 3 Operator Precedence Relations: - $a \lessdot b$: Terminal a yields precedence to terminal b (Shift b onto stack). - $a \doteq b$: Terminal a has same precedence as b (Part of same operator, e.g., (and)). - $a \gtrdot b$: Terminal a takes precedence over b (Reduce handle bounded by a).

🧠 M2 Active Recall & 10-Question University Exam Master Bank

Q1. Prove that merging CLR(1) states to form LALR(1) can NEVER produce a Shift-Reduce conflict. (8 Marks)

Suppose two CLR(1) states I_1 and I_2 have identical LR(0) cores. If an S-R conflict were to appear in the merged state $I_{1,2}$, there must exist an item $[A \to \alpha \cdot, a]$ and an item $[B \to \beta \cdot a\gamma, b]$.
Since shift actions depend strictly on the presence of the terminal a following the dot in the core $[B \to \beta \cdot a\gamma]$ (which is identical in both I_1 and I_2), the shift action was already present in both individual states.
The lookahead a for $[A \to \alpha \cdot]$ was present in either I_1 or I_2. Thus, the S-R conflict must have already existed in that original CLR(1) state!
Therefore, merging states with identical cores **can never introduce new Shift-Reduce conflicts** (though it can occasionally introduce Reduce-Reduce conflicts).

Q2. Calculate FIRST and FOLLOW sets for the following grammar and construct LL(1) parsing table: (10 Marks)
$S \to aBDh, \quad B \to cC, \quad C \to bC \mid \epsilon, \quad D \to EF, \quad E \to g \mid \epsilon, \quad F \to f \mid \epsilon$
FIRST Sets: $\text{FIRST}(S) = \{a\}, \text{FIRST}(B) = \{c\}, \text{FIRST}(C) = \{b, \epsilon\}, \text{FIRST}(D) = \{g, f, \epsilon\}, \text{FIRST}(E) = \{g, \epsilon\}, \text{FIRST}(F) = \{f, \epsilon\}$.
FOLLOW Sets: $\text{FOLLOW}(S) = \{\$\}, \text{FOLLOW}(B) = \{g, f, h\}, \text{FOLLOW}(C) = \{g, f, h\}, \text{FOLLOW}(D) = \{h\}, \text{FOLLOW}(E) = \{f, h\}, \text{FOLLOW}(F) = \{h\}$.
Table $M[A, a]$ has zero overlapping entries $\implies$ Grammar is LL(1).

Q3. Differentiate between Top-Down and Bottom-Up Parsing across 5 dimensions. (8 Marks)

1. **Starting Point:** Top-Down starts at Root (S); Bottom-Up starts at Leaves (input terminals).
2. **Derivation Strategy:** Top-Down constructs Leftmost Derivation; Bottom-Up constructs Rightmost Derivation in Reverse.
3. **Grammar Restrictions:** Top-Down cannot handle Left Recursion; Bottom-Up handles both Left and Right recursion seamlessly.
4. **Lookahead Power:** LL(1) sees 1 token lookahead; LR(1) can see handles after complete subtrees are built.
5. **Implementation:** Top-Down uses recursive descent or $M[A, a]$ stack; Bottom-Up uses Shift-Reduce DFA state tables.

Q4. What is a Handle in Shift-Reduce parsing? Why is Handle Pruning significant? (6 Marks)

A **Handle** is a substring β in a right-sentential form $\alpha\beta w$ that matches the right-hand side of a production $A \to \beta$ such that replacing β with A represents the reverse of the previous step in a rightmost derivation ($S \Rightarrow_{\text{rm}}^{*} \alpha A w \Rightarrow_{\text{rm}} \alpha\beta w$). **Handle Pruning** guarantees that the bottom-up parser reconstructs the exact valid parse tree from leaves to start symbol without dead-end backtracking.

Q5. Explain the Yacc / Bison LALR(1) Parser Generator Architecture and conflict resolution mechanisms. (8 Marks)

Yacc takes a grammar specification (`parser.y`) and constructs an LALR(1) state table. It resolves Shift-Reduce conflicts using default rules (favor Shift) or explicit `%left`, `%right`, `%nonassoc` precedence directives. It resolves Reduce-Reduce conflicts by reducing via the rule listed earliest in the specification file.

Topic 17.6: Complete LR(0), SLR(1), CLR(1), LALR(1) & Operator Precedence Traces

🏛️ **Step-by-Step Solved Problem: Canonical Collection of LR(0) Items for Grammar $S \to AA, A \to aA \mid b$**

Augmented Grammar: $S' \to S, \quad S \to AA, \quad A \to aA \mid b$

- **State** $I_0 = \text{Closure}(\{[S' \to \cdot S]\})$:

```
[S' → . S]
[S  → . A A]
[A  → . a A]
[A  → . b]
```

- $\text{Goto}(I_0, S) = I_1 : \{[S' \to S\cdot]\} \implies \text{ACCEPT}$
- $\text{Goto}(I_0, A) = I_2 : \{[S \to A \cdot A], [A \to \cdot aA], [A \to \cdot b]\}$
- $\text{Goto}(I_0, a) = I_3 : \{[A \to a \cdot A], [A \to \cdot aA], [A \to \cdot b]\}$
- $\text{Goto}(I_0, b) = I_4 : \{[A \to b\cdot]\} \implies \text{Reduce } A \to b$
- $\text{Goto}(I_2, A) = I_5 : \{[S \to AA\cdot]\} \implies \text{Reduce } S \to AA$
- $\text{Goto}(I_3, A) = I_6 : \{[A \to aA\cdot]\} \implies \text{Reduce } A \to aA$

🏛️ Operator Precedence Parsing Table Construction & Evaluation Trace

Grammar: $E \to E + E \mid E * E \mid \mathbf{id}$

Operator	$+$	$*$	$\mathbf{id}$	$
$+$	$\cdot >$	$< \cdot$	$< \cdot$	$\cdot >$
$*$	$\cdot >$	$\cdot >$	$< \cdot$	$\cdot >$
$\mathbf{id}$	$\cdot >$	$\cdot >$	Error	$\cdot >$
$	$< \cdot$	$< \cdot$	$< \cdot$	Accept

Stack Evaluation Trace for Input `id + id * id $`:

1. Stack: `$< \cdot\mathbf{id}$ | Input: `+ id * id $` $\implies Reduce \mathbf{id}$
2. Stack: `$< \cdot + < \cdot\mathbf{id}$ | Input: `* id $` $\implies Reduce \mathbf{id}$
3. Stack: `$< \cdot + < \cdot * < \cdot\mathbf{id}$ | Input: `$` $\implies Reduce \mathbf{id}$
4. Stack: `$< \cdot + < \cdot * $ | $Input:$ `$` $\implies * \cdot > \$ \implies$ Reduce $*$: Pop $*$
5. Stack: `$< \cdot + $ | $Input:$ `$` $\implies + \cdot > \$ \implies$ Reduce $+$: Pop $+$
6. Stack: `$` | $Input:$ `$` $\implies$ **ACCEPT!**

Q6. Compare LL(1), SLR(1), CLR(1), and LALR(1) parsers across 5 fundamental dimensions. (10 Marks)

1. **Parsing Strategy:** LL(1) is Top-Down; SLR, CLR, LALR are Bottom-Up Shift-Reduce.
2. **Grammar Power:** $\mathrm{LL}(1) \subset \mathrm{SLR}(1) \subset \mathrm{LALR}(1) \subset \mathrm{CLR}(1)$.
3. **State Count:** LL(1) uses non-terminal rows; LR(0)/SLR(1)/LALR(1) share identical K states; CLR(1) has $5 \times$ to $10 \times K$ states.
4. **Reduction Criteria:** SLR uses $\mathrm{FOLLOW}(A)$; CLR uses specific lookahead item $[A \to \alpha\cdot, a]$; LALR merges lookaheads for identical cores.
5. **Implementation:** LL(1) uses recursive descent or $M[A, a]$ stack; LR parsers use deterministic shift-reduce finite state tables.

Q7. What is an Ambiguous Grammar? Prove that the expression grammar $E \to E + E \mid E * E \mid \mathbf{id}$ is ambiguous. (8 Marks)

A grammar is **Ambiguous** if there exists at least one input string with ≥ 2 distinct parse trees or leftmost derivations.
For string `id + id * id`: Derivation 1 applies $E \to E + E$ first (yielding $+ > *$), while Derivation 2 applies $E \to E * E$ first (yielding $* > +$). Because two distinct parse trees exist with differing precedence interpretations, the grammar is strictly ambiguous!

Q8. State the conditions under which a context-free grammar is guaranteed to be LL(1). (6 Marks)

A grammar G is LL(1) if and only if for every non-terminal A with distinct productions $A \to \alpha \mid \beta$:
1. $\mathrm{FIRST}(\alpha) \cap \mathrm{FIRST}(\beta) = \emptyset$
2. If $\alpha \Rightarrow^* \epsilon$, then $\mathrm{FIRST}(\beta) \cap \mathrm{FOLLOW}(A) = \emptyset$
3. If $\beta \Rightarrow^* \epsilon$, then $\mathrm{FIRST}(\alpha) \cap \mathrm{FOLLOW}(A) = \emptyset$.

Q9. Explain the Closure and Goto functions used in constructing the canonical collection of LR items. (8 Marks)

• $\text{Closure}(\mathbf{I})$: If $[A \to \alpha \cdot B\beta] \in I$, add $[B \to \cdot\gamma]$ for every production $B \to \gamma$ in the grammar until no more items can be added.
• $\text{Goto}(\mathbf{I}, \mathbf{X})$: Defined as $\text{Closure}(\{[A \to \alpha X \cdot \beta] \mid [A \to \alpha \cdot X\beta] \in I\})$. Represents the target DFA state reached when grammar symbol X is shifted or transitioned on top of the parser stack.

Q10. Explain Error Recovery in Predictive LL(1) Parsers using Synchronizing Tokens and Panic Mode. (8 Marks)

When predictive parser table lookup $M[A, a]$ encounters an empty error entry:
1. **Synchronizing Tokens:** All terminal symbols in $\text{FOLLOW}(A)$ are designated as synchronizing tokens.
2. If token $a \in \text{FOLLOW}(A)$, the parser pops non-terminal A from the stack, assuming A has been satisfied, and resumes parsing.
3. If token $a \notin \text{FOLLOW}(A)$, the parser skips incoming token a from the input stream until a synchronizing token is reached.

Module III: Semantic Analysis & Intermediate Code Generation — 9-Topic Master Guide

Topic 18: Introduction to Semantic Analysis (Static vs. Dynamic Semantic Checks)

Topic 19: Syntax-Directed Definitions (SDD Attribute Grammars & Dependency Graphs)

Topic 20: Syntax-Directed Translation Schemes (SDTS Embedded Semantic Actions)

Topic 21: SDTS for Declaration Processing & Scoped Symbol Table Design

Topic 22: Three-Address Code (Quadruples, Triples, Indirect Triples & DAG IR)

Topic 23: Types of Attributes (Synthesized vs. Inherited & S-Attributed vs. L-Attributed)

Topic 24: Type Checking for Expressions (Type Equivalence, Inferences & Coercions)

Topic 25: Intermediate Code Generation for Complex Assignment Statements

Topic 26: Translation of Multi-Dimensional Array References (Row/Col Major Math)

Topic 18: Introduction to Semantic Analysis & Static Type Systems

Semantic Analysis constitutes the third major phase of a modern optimizing compiler. While syntax analysis ensures that a program conforms to the formal context-free grammar of the programming language (e.g., verifying matching parentheses, balanced statement delimiters, and correct keyword placement), it is mathematically incapable of enforcing contextual relationships—such as checking whether a variable is declared before its point of use, verifying that array index types match declared array bounds, or validating that operator argument signatures conform to type rules.

Semantic Check Category	Compiler Responsibility & Mechanism	Representative Error Scenarios
1. Static Semantic Checks *(Evaluated at Compile-Time)*	Executed directly by the compiler front-end by traversing the Abstract Syntax Tree (AST) with Symbol Table queries.	• Type mismatch: assigning a string literal to an integer scalar variable. • Undeclared identifier referenced in an arithmetic expression. • Duplicate identifier declared within the same scope block. • Function called with an incorrect number or types of actual arguments. • `break` or `continue` statement used outside of a loop or switch body.
2. Dynamic Semantic Checks *(Evaluated at Run-Time)*	Emitted by the compiler as runtime boundary assertions or trapped by target CPU hardware exceptions during execution.	• Arithmetic division by zero (`x / 0`). • Array index out of bounds (`arr[100]` on array of size 10). • Dereferencing a `NULL` or uninitialized memory pointer. • Stack overflow resulting from unbounded recursive procedure calls. • Dynamic type casting errors (e.g., `ClassCastException` in Java).

Topic 19 & 23: Syntax-Directed Definitions (SDD) & Attribute Grammars

A **Syntax-Directed Definition (SDD)** is a Context-Free Grammar augmented with attributes and semantic rules. An attribute is a quantity associated with a grammar symbol (such as a numerical value, a data type, a memory offset, a symbol table entry pointer, or a generated string of intermediate code).

Formal Classification: Synthesized vs. Inherited Attributes

- **Synthesized Attribute:** An attribute of a non-terminal node A in a parse tree is synthesized if its value is computed exclusively from the attribute values of the **children** of A:

$$A.s = f(Y_1.a, Y_2.a, \ldots, Y_k.a) \quad \text{for production } A \to Y_1 Y_2 \ldots Y_k$$

Synthesized attributes represent *bottom-up information flow* (from terminal leaves upward toward the parse tree root).

- **Inherited Attribute:** An attribute of a grammar symbol Y_j on the RHS of a production is inherited if its value is computed from the attribute values of the **parent** A and/or the **left siblings** $Y_1, \ldots, Y_{j-1}$:

$$Y_j.i = f(A.a, Y_1.a, \ldots, Y_{j-1}.a) \quad \text{for production } A \to Y_1 \ldots Y_j \ldots Y_k$$

Inherited attributes represent *top-down and left-to-right information flow* (such as propagating declared data types down to lists of variable identifiers).

19.1 S-Attributed vs. L-Attributed Definitions: Exhaustive Engineering Comparison

Evaluation Dimension	S-Attributed Definitions	L-Attributed Definitions
Permissible Attributes	Utilizes **ONLY Synthesized attributes**. Zero inherited attributes permitted.	Permits both **Synthesized attributes** AND **Inherited attributes** (subject to strict dependency ordering).
Dependency Direction	Dependencies flow strictly upward from child nodes to parent nodes.	Dependencies flow downward from parent to children, and strictly **from left to right** among sibling nodes.
Parse Tree Traversal	Evaluated in a single bottom-up Post-Order traversal of the parse tree.	Evaluated in a single Depth-First, Left-to-Right Pre-Order traversal.
Bottom-Up LR Parsing	Can be evaluated **on-the-fly during LR bottom-up parsing** by executing semantic actions upon reduction. Attribute values are held in an auxiliary parser stack.	Requires special transformations (such as inserting marker non-terminals ϵ-rules) to evaluate on-the-fly in bottom-up LR parsers.
Top-Down LL Parsing	Easily evaluated in LL parsers by passing synthesized attributes up as procedure return values.	**Perfect Natural Match:** Inherited attributes are passed down as function arguments; synthesized attributes returned as results.

Topic 20 & 21: Syntax-Directed Translation Schemes (SDTS) & Declarations

A **Syntax-Directed Translation Scheme (SDTS)** is a context-free grammar where program fragments called **semantic actions** are embedded directly within the right-hand sides of productions inside curly braces `{...}`.

🏛️ Step-by-Step SDTS: Processing Multi-Variable Declarations `int x, y, z;`

The compiler must propagate the base type `int` across an arbitrary sequence of declared variable identifiers:

```
D  → T { L.in = T.type } L
T  → int { T.type = integer }
T  → float { T.type = real }
L  → { L1.in = L.in } L1, id { enter_symbol_table(id.name, L.in) }
L  → id { enter_symbol_table(id.name, L.in) }
```

Complete Annotated Parse Tree Evaluation Walkthrough:

1. The parser reaches the leaf token `int` under non-terminal T. The semantic rule `{T.type = integer}` computes the synthesized attribute $T.\text{type} = \text{integer}$.
2. At the production $D \rightarrow T\ L$, the semantic action `{L.in = T.type}` assigns the inherited attribute $L.\text{in} = \text{integer}$ to child non-terminal L.
3. Node L expands to $L_1, \mathbf{id}_3$ (representing identifier `z`). It assigns $L_1.\text{in} = L.\text{in} = \text{integer}$ and invokes `enter_symbol_table("z", integer)`.
4. Node L_1 expands to $L_2, \mathbf{id}_2$ (representing identifier `y`). It assigns $L_2.\text{in} = L_1.\text{in} = \text{integer}$ and invokes `enter_symbol_table("y", integer)`.
5. Node L_2 expands to the base production $\mathbf{id}_1$ (representing identifier `x`). It executes `enter_symbol_table("x", integer)`.

Topic 22: Three-Address Code (TAC) & Intermediate Representations

Three-Address Code (TAC) is a linearized, machine-independent intermediate language where each statement contains at most one operator and at most three operands of the general form: $x = y\ \mathbf{op}\ z$.

Representation Form	Internal Structural Data Fields	Key Advantages & Tradeoffs
1. Quadruples	Explicit record containing 4 fields: `(op, arg1, arg2, result)`. Intermediate values are assigned explicit temporary variable names (`t1`, `t2`).	• Statements can be reordered freely by optimization passes. • Symbol Table is enlarged to track numerous compiler-generated temporary names.
2. Triples	Explicit record containing 3 fields: `(op, arg1, arg2)`. Results of computations are referenced implicitly by their array statement index `(0)`, `(1)`.	• Saves memory by eliminating temporary variable names. • Code reordering or optimization requires updating all dependent pointer index references.
3. Indirect Triples	Consists of a primary array of pointers indexing into a separate static table of Triples.	• **Best of Both Worlds:** Optimizers reorder, insert, or delete instructions simply by modifying the pointer array without rewriting Triples.

🏛️ Complete Comparative Implementation: Translation of `x = (a + b) * - c + (a + b)`

Index	Quadruple: (op, arg1, arg2, result)	Triple: (op, arg1, arg2)	Indirect Triple Pointer Array
(0)	`(+, a, b, t1)`	`(+, a, b)`	Statement `[0]` → Triple (0)
(1)	`(uminus, c, -, t2)`	`(uminus, c, -)`	Statement `[1]` → Triple (1)
(2)	`(*, t1, t2, t3)`	`(*, (0), (1))`	Statement `[2]` → Triple (2)
(3)	`(+, a, b, t4)`	`(+, a, b)`	Statement `[3]` → Triple (3)
(4)	`(+, t3, t4, t5)`	`(+, (2), (3))`	Statement `[4]` → Triple (4)
(5)	`(=, t5, -, x)`	`(=, x, (4))`	Statement `[5]` → Triple (5)

Topic 24 & 25: Type Checking & Type Conversions

Type Equivalence & Conversion Definitions: - **Name Equivalence:** Two types are considered equivalent if and only if their declared type names are identical. - **Structural Equivalence:** Two types are considered equivalent if they possess identical internal memory layouts, component counts, and field types (even under different type aliases). - **Type Coercion (Implicit Widening):** Automatic conversion performed by the compiler without explicit programmer syntax (e.g., promoting an `int` to a `float` in `float_var = int_var + 2.5`). - **Type Casting (Explicit Narrowing):** Programmer-directed type override (e.g., `(int) float_var`) which may incur precision truncation.

Topic 26: Translation of Multi-Dimensional Array References

While programming languages provide multi-dimensional coordinate arrays $A[i_1, i_2, \ldots, i_k]$, physical memory hardware is strictly a **one-dimensional linear byte array**. The compiler must synthesize Three-Address Code to calculate the linear memory offset at runtime.

Figure 3.1: Linear Memory Storage Conventions for Multi-Dimensional Arrays

26.1 Mathematical Derivation of Multi-Dimensional Array Addresses:

1. One-Dimensional Array ($A[\text{low}..\text{high}]$ with element width w):

$$\text{Address}(A[i]) = \text{base}_A + (i - \text{low}) \times w$$

2. Two-Dimensional Array ($A[l_1..h_1,\ l_2..h_2]$ in Row-Major Order):

$$\text{Address}(A[i_1, i_2]) = \text{base}_A + \Big[(i_1 - l_1) \times n_2 + (i_2 - l_2)\Big] \times w$$

Where $n_2 = (h_2 - l_2 + 1)$ represents the number of elements per row (number of columns).

3. Two-Dimensional Array ($A[l_1..h_1,\ l_2..h_2]$ in Column-Major Order):

$$\text{Address}(A[i_1, i_2]) = \text{base}_A + \Big[(i_2 - l_2) \times n_1 + (i_1 - l_1)\Big] \times w$$

Where $n_1 = (h_1 - l_1 + 1)$ represents the number of elements per column (number of rows).

4. Three-Dimensional Array ($A[l_1..h_1,\ l_2..h_2,\ l_3..h_3]$ in Row-Major Order):

$$\text{Address}(A[i_1, i_2, i_3]) = \text{base}_A + \Big[\big((i_1 - l_1) \times n_2 + (i_2 - l_2)\big) \times n_3 + (i_3 - l_3)\Big] \times w$$

Where $n_2 = (h_2 - l_2 + 1)$ and $n_3 = (h_3 - l_3 + 1)$.

5. General k-Dimensional Row-Major Linear Offset (Horner's Rule):

$$\text{Offset} = \Big(\ldots\big((i_1 - l_1) \cdot n_2 + (i_2 - l_2)\big) \cdot n_3 + \ldots\Big) \cdot n_k + (i_k - l_k)$$

$$\text{Address}(\mathbf{A}[\mathbf{i_1}, \ldots, \mathbf{i_k}]) = \text{base}_\mathbf{A} + \text{Offset} \times \mathbf{w}$$

🏛 Step-by-Step Solved Problem: Generating Three-Address Code for $X = A[i, j] + B[k, l]$

Let A be a 10×20 2D array of integers ($w = 4$ bytes) with 0-based indexing ($n_2 = 20$).
Let B be a 15×30 2D array of integers ($w = 4$ bytes) with 0-based indexing ($n_2 = 30$).

```
// 1. Calculate linear offset for array reference A[i, j]:
t1 = i * 20           ; (i * n2)
t2 = t1 + j           ; (i * n2 + j)
t3 = t2 * 4           ; Byte offset = element_offset * 4
t4 = A[t3]            ; Load value from A at byte offset t3

// 2. Calculate linear offset for array reference B[k, l]:
t5 = k * 30           ; (k * n2)
t6 = t5 + l           ; (k * n2 + l)
t7 = t6 * 4           ; Byte offset = element_offset * 4
t8 = B[t7]            ; Load value from B at byte offset t7

// 3. Perform addition and assign to scalar variable X:
t9 = t4 + t8
X  = t9
```

🧠 M3 Active Recall & 10-Question University Exam Master Bank

Q1. Explain Syntax-Directed Definitions (SDD) and differentiate between S-Attributed and L-Attributed definitions with grammar examples. (10 Marks)

An **SDD** is a context-free grammar augmented with attributes associated with grammar symbols and semantic rules associated with productions.
- **S-Attributed Definitions:** Contain exclusively Synthesized attributes, where a node's value is computed strictly from its children: $A.s = f(Y_1.a, \ldots, Y_k.a)$. They can be evaluated naturally during bottom-up LR parsing on the reduction stack.
Example: $E \to E_1 + T \ \{E.\text{val} = E_1.\text{val} + T.\text{val}\}$.
- **L-Attributed Definitions:** Permit both Synthesized and Inherited attributes, but inherited attributes of a RHS symbol Y_j can only depend on inherited attributes of parent A or attributes of left siblings $Y_1 \ldots Y_{j-1}$. They can be evaluated during top-down LL parsing in a single depth-first pass.
Example: $D \to T \ \{L.\text{in} = T.\text{type}\} \ L$.

Q2. Derive the 2D array row-major address calculation formula and generate Three-Address Code for $A[i, j] = B[i, j] + C[i]$. (10 Marks)

Row-Major 2D Address Formula: $\text{Address}(A[i, j]) = \text{base}_A + [(i - l_1) \cdot n_2 + (j - l_2)] \cdot w$.
Generated Three-Address Code Sequence (Assuming 0-based indexing and width $w = 4$):

```
t1 = i * n2_B
t2 = t1 + j
t3 = t2 * 4
t4 = B[t3]          ; Load B[i, j]
t5 = i * 4
t6 = C[t5]          ; Load C[i]
t7 = t4 + t6        ; Sum
t8 = i * n2_A
t9 = t8 + j
t10 = t9 * 4
A[t10] = t7         ; Store into A[i, j]
```

Q3. Compare Quadruples, Triples, and Indirect Triples representations of Three-Address Code across 4 dimensions. (8 Marks)

1. **Field Count:** Quadruples have 4 fields `(op, arg1, arg2, res)`; Triples have 3 fields `(op, arg1, arg2)`; Indirect Triples use an auxiliary pointer array to index into 3-field Triples.
2. **Temporary Names:** Quadruples generate explicit temporaries (`t1`, `t2`); Triples refer to statement indices `(0)`, `(1)`.
3. **Reordering Overhead:** Quadruples can be reordered freely without rewriting; Triples require rewriting all index pointers; Indirect Triples reorder pointers without touching triple records.
4. **Memory Consumption:** Triples are most compact; Quadruples consume extra symbol table memory.

Q4. What is Type Coercion? How does the compiler handle implicit vs explicit type conversions? (6 Marks)

Type Coercion is the automatic conversion of a value from one data type to another performed by the compiler (e.g., widening an `int` to a `float` in `x = float_val + int_val` by inserting `inttofloat`). **Explicit Type Casting** is programmer-specified (e.g., `(int) float_val`), forcing narrowing conversions that may truncate precision.

Q5. Explain the Scoped Symbol Table implementation for nested blocks in C/C++. (8 Marks)

Nested scopes are represented either via a **Tree of Symbol Tables** (where each child scope points to its parent scope via an `enclosing_scope` pointer) or via a **Scoped Hash Table with Scope Markers**. When entering a new block `{`, a new scope level is pushed onto the scope stack. When leaving a block `}`, all variables declared at the current scope level are unlinked and popped from the hash buckets, ensuring outer shadow variables become visible again in $O(1)$ time.

Topic 26.2: Comprehensive Worked Numerical Problems in Intermediate Code & SDD

🏛️ Solved Numerical 1: Row-Major Address Calculation for 3D Tensor Array

Let A be a 3D array declared as $A[1..10,\ 1..20,\ 1..30]$ of 4-byte integers with base address $\text{base} = 2000$.
Find the exact physical byte address of element $A[4, 12, 18]$ in Row-Major order.

Solution:

$$\text{Bounds: } l_1 = 1, h_1 = 10, \quad l_2 = 1, h_2 = 20, \quad l_3 = 1, h_3 = 30$$

$$\text{Dimension sizes: } n_2 = (20 - 1 + 1) = 20, \quad n_3 = (30 - 1 + 1) = 30$$

$$\text{Offset} = \big[(4 - 1) \times 20 + (12 - 1)\big] \times 30 + (18 - 1)$$

$$\text{Offset} = \big[3 \times 20 + 11\big] \times 30 + 17 = [60 + 11] \times 30 + 17 = 71 \times 30 + 17 = 2130 + 17 = 2147$$

$$\text{Address}(\mathbf{A}[\mathbf{4, 12, 18}]) = \mathbf{2000} + \mathbf{2147} \times \mathbf{4} = \mathbf{2000} + \mathbf{8588} = \mathbf{10588}$$

🏛️ Solved Numerical 2: Column-Major Address Calculation for 2D Array

Let B be a 2D array declared as $B[-5..15,\ 2..10]$ with base address $\text{base} = 5000$ and element width $w = 8$ bytes.
Calculate the exact physical byte address of element $B[3, 7]$ in Column-Major order.

Solution:

$$\text{Bounds: } l_1 = -5, h_1 = 15 \implies n_1 = (15 - (-5) + 1) = 21, \quad l_2 = 2, h_2 = 10$$

$$\text{Column-Major Formula: } \text{Address}(B[i, j]) = \text{base} + \big[(j - l_2) \times n_1 + (i - l_1)\big] \times w$$

$$\text{Offset} = (7 - 2) \times 21 + (3 - (-5)) = 5 \times 21 + 8 = 105 + 8 = 113$$

$$\text{Address}(\mathbf{B}[\mathbf{3, 7}]) = \mathbf{5000} + \mathbf{113} \times \mathbf{8} = \mathbf{5000} + \mathbf{904} = \mathbf{5904}$$

🏛️ Solved Numerical 3: Syntax-Directed Definition for Desktop Arithmetic Calculator

Production	Semantic Rule	Attribute Type
$L \to E\ \mathbf{n}$	$\text{print}(E.\text{val})$	Synthesized
$E \to E_1 + T$	$E.\text{val} = E_1.\text{val} + T.\text{val}$	Synthesized
$E \to T$	$E.\text{val} = T.\text{val}$	Synthesized
$T \to T_1 * F$	$T.\text{val} = T_1.\text{val} \times F.\text{val}$	Synthesized
$T \to F$	$T.\text{val} = F.\text{val}$	Synthesized
$F \to (E)$	$F.\text{val} = E.\text{val}$	Synthesized
$F \to \mathbf{digit}$	$F.\text{val} = \mathbf{digit}.\text{lexval}$	Synthesized

Q6. Differentiate between Synthesized and Inherited attributes with concrete annotated parse tree examples. (8 Marks)

• **Synthesized Attributes:** Computed strictly from children nodes ($A.s = f(C_1.a, \ldots, C_k.a)$). Evaluated bottom-up in Post-Order traversal (e.g. arithmetic expression values $E.\text{val} = E_1.\text{val} + T.\text{val}$).
• **Inherited Attributes:** Computed from parent node or left siblings ($Y.i = f(\text{Parent}.a, \text{Sibling}.a)$). Evaluated top-down in Pre-Order traversal (e.g. data type propagation in declarations $L.\text{in} = T.\text{type}$).

Q7. What is a Dependency Graph in Syntax-Directed Definitions? How is Topological Sorting used to evaluate attributes? (10 Marks)

A **Dependency Graph** is a directed graph where nodes represent attribute instances at parse tree nodes, and a directed edge from $u \to v$ indicates that attribute v depends on attribute u ($v = f(u, \ldots)$). A valid evaluation order exists if and only if the dependency graph contains zero directed cycles. A **Topological Sort** of the DAG produces a linear evaluation sequence ensuring every attribute is computed before its consumers read it!

Q8. Explain Type Checking and Type Inferences for Expressions with formal typing rules. (8 Marks)

A **Type System** defines rules assigning types to programming constructs. A formal typing rule is expressed as:

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}}$$

Where Γ represents the typing environment (Symbol Table). If operand types differ (e.g. $\text{float} + \text{int}$), the type inference engine checks for valid implicit coercions ($\text{int} \to \text{float}$) before failing with a semantic type error.

Q9. Generate Three-Address Code, Quadruples, Triples, and Indirect Triples for $a = b * -c + b * -c$. (10 Marks)

Three-Address Code:
`t1 = -c`, `t2 = b * t1`, `t3 = -c`, `t4 = b * t3`, `t5 = t2 + t4`, `a = t5`.
Quadruples: `(uminus, c, -, t1)`, `(*, b, t1, t2)`, `(uminus, c, -, t3)`, `(*, b, t3, t4)`, `(+, t2, t4, t5)`, `(=, t5, -, a)`.
Triples: (0): `(uminus, c, -)`, (1): `(*, b, (0))`, (2): `(uminus, c, -)`, (3): `(*, b, (2))`, (4): `(+, (1), (3))`, (5): `(=, a, (4))`.

Q10. Explain the role of Abstract Syntax Trees (AST) vs Concrete Parse Trees in Semantic Analysis. (6 Marks)

A **Concrete Parse Tree** contains every grammatical terminal and non-terminal used during parsing, including punctuation tokens, parentheses, and single-production chains ($E \to T \to F$). An **Abstract Syntax Tree (AST)** compresses the parse tree into an operator-operand hierarchy where operators become interior nodes and operands become leaves, discarding redundant formatting tokens and dramatically accelerating semantic passes.

Topic 26.3: Advanced Type Checking, Equivalence & AST Linearization

A compiler's type checker enforces the type system rules of the source programming language. The type system assigns **Type Expressions** to program elements. A type expression is either a basic type (such as `integer`, `float`, `char`, `type_error`, `void`) or formed by applying a **Type Constructor**:

Array Type Constructor: If T is a type expression, then $\text{array}(I, T)$ is a type expression denoting an array with index set I and elements of type T.
Product Type Constructor: If T_1 and T_2 are type expressions, then their Cartesian product $T_1 \times T_2$ represents pairs of elements.
Record / Structure Constructor: Encapsulates named field elements: $\text{record}((f_1 \times T_1) \times \cdots \times (f_k \times T_k))$.
Function Type Constructor: Maps domain types to range types: $T_1 \to T_2$.
Pointer Constructor: If T is a type, then $\text{pointer}(T)$ represents memory address locations referencing values of type T.

🏛️ Step-by-Step Solved Problem: Type Checking Rules & Structural Equivalence Verification

Consider the following C type definitions:

```
typedef struct { int x; float y; } PointA;
typedef struct { int x; float y; } PointB;
PointA p1;
PointB p2;
```

Evaluation:

- **Under Name Equivalence:** `p1` and `p2` have **DIFFERENT** types because their declared type names (`PointA` vs `PointB`) differ. An assignment `p1 = p2` results in a compile-time static type error!
- **Under Structural Equivalence:** `p1` and `p2` have **IDENTICAL** types because their internal memory structures are both $\text{record}((\text{"x"} \times \text{int}) \times (\text{"y"} \times \text{float}))$. The assignment `p1 = p2` is permitted!

🏛️ Complete Step-by-Step Translation of Array Assignment Statement with Scaling

Source Statement: `A[i][j] = B[i][j] * C[k] + 25.5`

```
// 1. Calculate address of B[i][j] (Dimensions: 10 x 20, element width: 8 bytes)
t1 = i * 20
t2 = t1 + j
t3 = t2 * 8
t4 = B[t3]            ; Load B[i][j] float value into temporary t4

// 2. Calculate address of C[k] (1D Array, element width: 8 bytes)
t5 = k * 8
t6 = C[t5]            ; Load C[k] float value into temporary t6

// 3. Multiply and Add Constant
t7 = t4 * t6          ; Product B[i][j] * C[k]
t8 = t7 + 25.5        ; Add floating-point constant literal

// 4. Calculate target address of A[i][j] (Dimensions: 10 x 20, element width: 8 bytes)
t9 = i * 20
t10 = t9 + j
t11 = t10 * 8
A[t11] = t8           ; Store computed value into A[i][j]
```

Module IV: Control Flow, Backpatching & Runtime Environments — 9-Topic Master Guide

Topic 27: Intermediate Code for Boolean Expressions (Short-Circuit Evaluation)

Topic 28: Control Flow Translation (`if-then`, `if-then-else`, `while`, `for`, `switch`)

Topic 29: Backpatching Mechanics (`makelist`, `merge`, `backpatch`, `truelist`, `falselist`)

Topic 30: Translation of Forward & Backward Jump Control Structures

Topic 31: Translation of Procedure Calls & Returns (`param`, `call`, `return`)

Topic 32: Runtime Storage Organization (Code, Static, Heap & Call Stack)

Topic 33: Activation Records Anatomy (The 7 P-R-C-A-S-L-T Frame Fields)

Topic 34: Storage Allocation Strategies (Static, Stack & Heap Allocation)

Topic 35: Access to Non-Local Variables (Access Links vs. Displays Technique)

Topic 27 & 28: Boolean Expressions & Control Flow Translation

In programming languages, **Boolean Expressions** serve two fundamentally distinct operational purposes:

Numerical Value Representation: Computing explicit logical truth values stored into boolean variables (e.g., `flag = (a > b)` where `flag` evaluates to integer `1` or `0`).
Flow-of-Control Branching: Governing the conditional execution trajectory in decision and looping statements (e.g., `if (a > b) ...` or `while (x <= y) ...`).

Short-Circuit Evaluation Semantics

Compilers generate highly optimized short-circuit conditional code that avoids evaluating redundant subexpressions:

- In an **OR expression** (B_1 **or** B_2): If B_1 evaluates to **true**, the overall expression is immediately known to be **true**; evaluation of B_2 is completely bypassed.
- In an **AND expression** (B_1 **and** B_2): If B_1 evaluates to **false**, the overall expression is immediately known to be **false**; evaluation of B_2 is completely bypassed.

Step-by-Step Solved Problem: Generating Three-Address Code for Complex Control Flow

Source Statement: `if (a < b || c < d && e < f) x = y + 1; else x = y - 1;`

```
100: if a < b goto 104      ; Short-circuit OR: if (a < b) is TRUE → jump directly to true-body
101: if c < d goto 102      ; If (c < d) is TRUE → must evaluate (e < f)
102: goto 107               ; If (c < d) is FALSE → short-circuit AND fails → jump to else-body
103: if e < f goto 104      ; If (e < f) is TRUE → jump to true-body
104: goto 107               ; If (e < f) is FALSE → jump to else-body
105: t1 = y + 1             ; True-body statement
106: x = t1
107: goto 110               ; Jump past else-body
108: t2 = y - 1             ; Else-body statement
109: x = t2
110:  ...                   ; Next statement
```

28.1 Translation of `switch-case` Statements: 3 Compilation Strategies

Compilation Strategy	Operating Mechanism	Ideal Scenario & Time Complexity
1. Linear Comparison Sequence	Emits sequential `if (E == V1) goto L1; if (E == V2) goto L2; ...`	Small number of case branches (≤ 4 cases); $O(n)$ search time.
2. Jump Table (Direct Indexing)	Creates a contiguous array of code pointers indexed directly by value $(E - V_{\min})$.	Dense, tightly clustered case values ($[1, 2, 3, 4, 5]$); $\mathbf{O(1)}$ instant jump time.
3. Binary Search Tree (Hash Table)	Emits a balanced binary search decision tree of conditional branches over sorted case values.	Large, sparse case values ($[10, 500, 10000, 500000]$); $O(\log n)$ search time.

Topic 29 & 30: Backpatching Mechanics in Single-Pass Compilers

In a single-pass compiler, when the code generator emits a conditional or unconditional branch instruction, the **target jump address** is frequently unknown because the target statement has not yet been parsed. **Backpatching** resolves this problem by storing lists of instruction indices requiring branch targets, and filling in (backpatching) the actual statement labels once they become known.

> **The 3 Fundamental Backpatching Helper Functions:** 1. $\mathbf{makelist}(i)$: Constructs a newly initialized list containing exactly one element: the integer index i of a newly emitted branch instruction. 2. $\mathbf{merge}(p_1, p_2)$: Concatenates two jump index lists p_1 and p_2, returning a unified combined list pointer. 3. $\mathbf{backpatch}(p, L)$: Traverses every instruction index stored in list p, inserting statement label L as the explicit target jump address.

29.1 Syntax-Directed Translation Scheme for Boolean Backpatching:

Grammar Production	Semantic Action Rule for Backpatching
$B \rightarrow B_1 \textbf{ or } M \ B_2$	$\text{backpatch}(B_1.\text{falselist}, M.\text{quad})$ $B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist})$ $B.\text{falselist} = B_2.\text{falselist}$
$B \rightarrow B_1 \textbf{ and } M \ B_2$	$\text{backpatch}(B_1.\text{truelist}, M.\text{quad})$ $B.\text{truelist} = B_2.\text{truelist}$ $B.\text{falselist} = \text{merge}(B_1.\text{falselist}, B_2.\text{falselist})$
$B \rightarrow \textbf{not } B_1$	$B.\text{truelist} = B_1.\text{falselist}$ $B.\text{falselist} = B_1.\text{truelist}$
$B \rightarrow (B_1)$	$B.\text{truelist} = B_1.\text{truelist}$ $B.\text{falselist} = B_1.\text{falselist}$
$B \rightarrow \mathbf{id}_1 \textbf{ relop } \mathbf{id}_2$	$B.\text{truelist} = \text{makelist}(\text{nextquad})$ $B.\text{falselist} = \text{makelist}(\text{nextquad} + 1)$ $\text{emit}(\text{"if "} \mathbf{id}_1.\text{name} \ \mathbf{relop}.\text{op} \ \mathbf{id}_2.\text{name} \ \text{"goto \_"})$ $\text{emit}(\text{"goto \_"})$
$M \rightarrow \epsilon$	$M.\text{quad} = \text{nextquad}$

Topic 31: Translation of Procedure Calls & Parameter Passing

Procedure calls require coordinating data passing, machine state preservation, stack frame allocation, and execution transfer between the **Caller** and **Callee** routines:

Three-Address Code Sequence for Procedure Call `result = compute_sum(a + 10, b * 2)`

```
t1 = a + 10
param t1           ; Push actual argument 1 onto parameter list
t2 = b * 2
param t2           ; Push actual argument 2 onto parameter list
call compute_sum, 2 ; Transfer control to procedure compute_sum (2 parameters)
result = retval    ; Retrieve return value from CPU return register
```

Parameter Passing Method	Operating Mechanism	Key Semantic Characteristics
1. Call-by-Value (C, Java Primitives)	Actual argument expression is evaluated and a local copy of the r-value is passed into the callee's stack frame.	Modifications inside the callee have zero effect on the caller's variable.
2. Call-by-Reference (C++ `&`, Fortran)	The l-value (memory address) of the actual argument is passed directly.	Changes made to the formal parameter immediately mutate the caller's actual variable.
3. Call-by-Restore / Copy-In Copy-Out	Actual value is copied in at function entry; final value is copied back out to caller's variable upon function return.	Behaves like call-by-reference except in cases of variable aliasing.
4. Call-by-Name (Algol 60)	The actual argument expression is substituted textually into the callee body as a parameterless function (called a **Thunk**).	Re-evaluates the argument expression every single time the formal parameter is accessed.

Topic 32 & 33: Runtime Storage Organization & Activation Records

The operating system allocates a continuous block of virtual memory to the executing process, organized into four canonical logical memory segments:

Figure 4.1: Canonical Process Virtual Memory Runtime Layout

33.1 Exhaustive Anatomy of an Activation Record (Stack Frame):

Activation Record Field	Internal Purpose & Operational Contents	Managing Party
1. Actual Parameters	Holds argument values passed from the caller to the callee.	Caller Routine
2. Returned Value	Reserved memory cell where the callee writes the function return result.	Callee Routine
3. Control Link (Dynamic Link)	Pointer pointing to the base address of the **caller's activation record**. Used to restore caller's stack frame upon return.	Callee Routine
4. Access Link (Static Link)	Pointer pointing to the activation record of the **lexical enclosing parent**. Used to resolve non-local variable references.	Callee Routine
5. Saved Machine Status	Preserves CPU Program Counter (PC), CPU registers, stack base pointers, and condition flags prior to procedure invocation.	Callee Routine
6. Local Variables	Stores local scalar variables and fixed-size arrays declared within the procedure body.	Callee Routine
7. Temporaries	Holds intermediate values generated during expression evaluation, array address offsets, and register spills.	Callee Routine

🧠 **Memory Hook: The 7 Fields of an Activation Record (P-R-C-A-S-L-T)**

Parameters → **R**eturned value → **C**ontrol link → **A**ccess link → **S**aved status → **L**ocal data → **T**emporaries

Topic 34 & 35: Storage Allocation & Non-Local Data Access (Displays)

Storage Strategy	Operating Mechanism	Advantages & Critical Tradeoffs
1. Static Allocation	All memory addresses for all variables and procedures are permanently fixed at compile time (Fortran 77).	Zero runtime allocation overhead; cannot support recursion or dynamic data structures.
2. Stack Allocation	Stack frames are pushed onto the call stack upon procedure call and popped upon procedure return (LIFO).	Supports unbounded recursion; local variable lifetimes strictly bounded by function lifetime.
3. Heap Allocation	Dynamically allocated in arbitrary order (`malloc`/`new`) and reclaimed via explicit `free` or Garbage Collection.	Enables dynamic data structures (trees, graphs); incurs memory fragmentation and allocation overhead.

Non-Local Variable Access: Static Links vs. Displays Array Technique

- **Access Links (Static Links):** Chained pointer traversal. Accessing a non-local variable defined k lexical nesting levels away requires dereferencing k successive pointers ($O(k)$ time complexity).
- **Displays Array:** An auxiliary global array of pointers `d[0..max_depth]` where `d[i]` directly points to the most recently active activation record at lexical nesting depth i. Provides guaranteed $\mathbf{O(1)}$ instant access to any non-local variable at any nesting depth!

🧠 M4 Active Recall & 10-Question University Exam Master Bank

Q1. Explain Backpatching with complete Syntax-Directed Translation Scheme for a `while (B) do S` loop. (10 Marks)

Production: $S \rightarrow \textbf{while}\ M_1\ B\ \textbf{do}\ M_2\ S_1$
Semantic Action Rules:

$$\text{backpatch}(S_1.\text{nextlist}, M_1.\text{quad}); \quad // \text{ Loop body branches back to condition}$$

$$\text{backpatch}(B.\text{truelist}, M_2.\text{quad}); \quad // \text{ True condition branches into loop body}$$

$$S.\text{nextlist} = B.\text{falselist}; \quad // \text{ False condition exits while loop}$$

$$\text{emit}(\text{"goto "}M_1.\text{quad}); \quad // \text{ Emit unconditional loop back-edge}$$

Q2. Draw and explain the complete anatomy of an Activation Record (Stack Frame) with all 7 fields. (10 Marks)

An Activation Record manages the execution state of a procedure instance. The 7 standard fields are:
1. **Actual Parameters:** Arguments passed into the procedure.
2. **Returned Value:** Result returned to caller.
3. **Control Link (Dynamic Link):** Pointer to caller's activation record for stack unwinding.
4. **Access Link (Static Link):** Pointer to enclosing static scope for non-local variables.
5. **Saved Machine Status:** Saved Program Counter, CPU registers, and status flags.
6. **Local Data:** Local variables declared in procedure body.
7. **Temporaries:** Temporary evaluation cells for complex expressions.

Q3. Compare Call-by-Value, Call-by-Reference, and Call-by-Name parameter passing methods with code examples. (8 Marks)

- **Call-by-Value:** R-value copied to local parameter. Changes inside callee do not affect caller.
- **Call-by-Reference:** L-value (address) passed directly. Changes inside callee immediately mutate caller's variable.
- **Call-by-Name:** Argument substituted textually as a parameterless Thunk function. Re-evaluated upon every access.

Q4. How does the Displays array technique achieve O(1) access to non-local variables compared to static links? (8 Marks)

Static links require traversing a linked list of k parent pointers ($O(k)$ time). The **Displays** technique maintains a global array `d[i]` pointing directly to the active frame at lexical depth i. Any non-local variable at depth i is accessed in single-step dereference `d[i]->offset`, achieving guaranteed $\mathbf{O(1)}$ lookup time!

Q5. Explain the Caller and Callee Calling Sequences and register saving conventions in x86-64 / MIPS. (8 Marks)

- **Caller Calling Sequence:** (1) Evaluates actual arguments; (2) Pushes caller-saved registers; (3) Emits `param` instructions; (4) Emits `call` which pushes return address.
- **Callee Prologue:** (1) Saves old frame pointer (`push rbp`); (2) Sets new frame pointer (`mov rbp, rsp`); (3) Allocates stack space for local variables (`sub rsp, framesize`); (4) Saves callee-saved registers.
- **Callee Epilogue:** (1) Writes return value to RAX register; (2) Restores callee-saved registers; (3) Restores stack pointer (`mov rsp, rbp; pop rbp`); (4) Emits `ret`.

Topic 35.2: Comprehensive Worked Control Flow & Runtime Numerical Traces

🏛️ Solved Problem 1: Complete Three-Address Code for Nested While Loop

Source Statement:

```
while (i < 10 && j > 0) {
    if (a[i] == b[j]) {
        x = x + 1;
    } else {
        y = y + 1;
    }
    i = i + 1;
    j = j - 1;
}
```

Generated Three-Address Code:

```
100: if i < 10 goto 102
101: goto 120              ; Exit loop if (i < 10) is False
102: if j > 0 goto 104     ; Check second condition
103: goto 120              ; Exit loop if (j > 0) is False
104: t1 = i * 4            ; Array offset a[i]
105: t2 = a[t1]
106: t3 = j * 4            ; Array offset b[j]
107: t4 = b[t3]
108: if t2 == t4 goto 111
109: goto 114              ; Branch to else-body
110: goto 114
111: t5 = x + 1            ; True-body: x = x + 1
112: x = t5
113: goto 116
114: t6 = y + 1            ; Else-body: y = y + 1
115: y = t6
116: t7 = i + 1            ; Loop step: i = i + 1
117: i = t7
118: t8 = j - 1            ; Loop step: j = j - 1
119: j = t8
120: goto 100              ; Loop back-edge to condition
121:  ...                  ; Next statement
```

🏛️ Solved Problem 2: Displays Array Non-Local Variable Access Execution Trace

Consider lexical nesting levels: `Main` (Depth 1) $\rightarrow$ `Procedure P` (Depth 2) $\rightarrow$ `Procedure Q` (Depth 3).

Displays Array Structure:

- `d[1]` points to active Activation Record of `Main`
- `d[2]` points to active Activation Record of `Procedure P`
- `d[3]` points to active Activation Record of `Procedure Q`

When `Procedure Q` references variable `x` declared in `Main` (at offset 16 from `Main`'s FP):

$$\text{Memory Target Address} = \mathbf{d[1]} + \mathbf{16} \quad (\text{Direct } \mathbf{O(1)} \text{ Instant Lookup!})$$

When `Procedure Q` calls `Procedure R` at depth 2, the old `d[2]` pointer is saved in `R`'s frame and `d[2]` is updated to point to `R`. Upon return, `d[2]` is restored to `P`.

Q6. Compare Static, Stack, and Heap Memory Allocation strategies across 5 dimensions. (10 Marks)

1. **Allocation Timing:** Static is fixed at compile-time; Stack occurs upon function entry (LIFO); Heap occurs on dynamic programmer request (`malloc`).
2. **Recursion Support:** Static cannot support recursion; Stack handles unbounded recursion seamlessly; Heap supports dynamic object graph structures.
3. **Data Lifetime:** Static persists for entire program run; Stack is strictly bounded by function return; Heap persists until explicit `free` or Garbage Collection.
4. **Overhead:** Static has zero runtime overhead; Stack has minimal pointer adjustment (`sub rsp, framesize`); Heap incurs memory fragmentation and allocator search costs.
5. **Deallocation:** Static is reclaimed on program termination; Stack automatically pops on return; Heap requires explicit deallocation or Garbage Collection.

Q7. Explain the Control Link (Dynamic Link) vs. Access Link (Static Link) in Activation Records. (8 Marks)

• **Control Link (Dynamic Link):** Points to the base address of the **caller's activation record**. It reflects the dynamic call history and is used to restore the caller's stack frame pointer upon function return.
• **Access Link (Static Link):** Points to the activation record of the **lexical enclosing parent procedure** in the source code. It reflects static lexical nesting and is used to resolve non-local variable references at runtime.

Q8. Explain Short-Circuit Boolean Evaluation with code generation templates for `if (B) S1 else S2`. (8 Marks)

Short-circuit evaluation stops evaluating boolean expressions as soon as the final truth value is determined.
Template for `if (B) S1 else S2`:
1. Translate boolean condition B, generating `B.truelist` and `B.falselist`.
2. $\text{backpatch}(B.\text{truelist}, \text{label}(S_1))$.
3. $\text{backpatch}(B.\text{falselist}, \text{label}(S_2))$.
4. Emit unconditional `goto` at the end of S_1 to jump past S_2.

Q9. How are `switch-case` statements compiled into Three-Address Code? Compare Jump Tables vs. Binary Search Trees. (8 Marks)

• **Linear Comparison Sequence:** Emits sequential `if (E == V) goto L` for ≤ 4 cases ($O(n)$ time).
• **Jump Table (Direct Array Indexing):** Creates an array of branch targets indexed by $(E - V_{\min})$ for dense integer ranges ($O(1)$ time).
• **Binary Search Decision Tree:** Emits a balanced binary search comparison tree for large, sparse case values ($O(\log n)$ time).

Q10. Explain the Garbage Collection Mark-and-Sweep algorithm for Heap Memory Management. (8 Marks)

The **Mark-and-Sweep** algorithm reclaims unreachable dynamic heap memory in 2 phases:
1. **Mark Phase:** Starts from Root Set (global variables, stack pointers, CPU registers) and performs a Graph Traversal (DFS/BFS), setting a `marked = true` bit on all reachable heap objects.
2. **Sweep Phase:** Linearly scans the entire heap space. Unmarked blocks are added to the Free List for future allocation; marked blocks have their mark bits cleared for the next cycle!

Topic 35.3: Advanced Runtime Memory Management & Parameter Passing Traces

In modern runtime environments, memory allocation and procedure execution depend heavily on the target hardware architecture, CPU calling conventions (such as System V AMD64 ABI), and memory management strategies.

Memory Arena	Hardware Characteristics	Growth Direction	Management Protocol
Code / Text	Read-only, executable pages mapped directly from executable binary.	Static fixed bounds	Operating System kernel memory pager.
Static / Data	Read-write, global and static variables initialized at compile time.	Static fixed bounds	Operating System loader initialization.
Heap Segment	Dynamic memory allocated via system calls (`sbrk`, `mmap`).	Grows Upward (↑)	`malloc`/`free` allocator with Buddy System.
Call Stack	LIFO stack frames allocated and freed on function entry and exit.	Grows Downward (↓)	Hardware Stack Pointer (`rsp`) & Frame Pointer (`rbp`).

🏛️ Step-by-Step Solved Problem: Complete Calling Sequence Machine Code Flow

Consider a function call `result = calculate(arg1, arg2)` on x86-64 Architecture:

```
; 1. CALLER SEQUENCE:
MOV RDI, [rbp - 8]     ; Place actual argument 1 into 1st argument register RDI
MOV RSI, [rbp - 16]    ; Place actual argument 2 into 2nd argument register RSI
CALL calculate         ; Pushes return address (RIP) to stack, jumps to calculate

; 2. CALLEE PROLOGUE:
calculate:
PUSH RBP               ; Save dynamic control link (caller's base pointer)
MOV RBP, RSP           ; Establish new stack frame base pointer
SUB RSP, 32            ; Allocate 32 bytes on stack for local variables and temporaries

; 3. CALLEE BODY & EPILOGUE:
; ... execute procedure instructions ...
MOV RAX, [rbp - 4]     ; Write return value into RAX register
MOV RSP, RBP           ; Deallocate local frame variables
POP RBP                ; Restore caller's base pointer (control link)
RET                    ; Pops return address from stack into RIP register

; 4. CALLER RESUMPTION:
MOV [rbp - 24], RAX    ; Store returned value from RAX into variable result
```

Topic 35.4: Comprehensive Calling Conventions & Dynamic Heap Allocators

A **Calling Convention** defines the low-level binary contract between caller and callee subroutines. It specifies how parameters are passed (registers vs. stack), how return values are delivered, which CPU registers must be preserved across calls (callee-saved vs. caller-saved), and who cleans up the parameter stack space.

Calling Convention	Parameter Passing Mechanism	Stack Cleanup Responsibility	Representative Platforms
cdecl (Standard C)	All parameters pushed onto stack right-to-left.	**Caller** cleans stack (`add rsp, N`).	x86 32-bit Linux / Windows C compilers.
stdcall	All parameters pushed onto stack right-to-left.	**Callee** cleans stack (`ret N`).	Windows Win32 API functions.
fastcall	First 2 arguments passed in `ECX`, `EDX`; remainder on stack.	**Callee** cleans stack.	Performance-critical x86 libraries.
System V AMD64 ABI	First 6 integer/pointer args in `RDI`, `RSI`, `RDX`, `RCX`, `R8`, `R9`; floats in `XMM0..7`.	**Caller** cleans stack frame.	x86-64 Linux, macOS, FreeBSD, GCC, Clang.

🏛️ Step-by-Step Solved Problem: Buddy System Dynamic Memory Allocation & Merging

Suppose a runtime heap starts with a single continuous block of size **1024 KB** (addressed from 0 to 1023). Trace allocations and deallocations:

1. **Request A (Alloc 70 KB):** Smallest power of 2 is $128\ \text{KB}$. The $1024\ \text{KB}$ block splits into two $512\ \text{KB}$ buddies $\rightarrow$ split to $256\ \text{KB} \rightarrow$ split to two $128\ \text{KB}$ blocks. Block $[0..127]$ is allocated to A.
2. **Request B (Alloc 200 KB):** Smallest power of 2 is $256\ \text{KB}$. Block $[256..511]$ is allocated to B.
3. **Request C (Alloc 110 KB):** Smallest power of 2 is $128\ \text{KB}$. Remaining buddy block $[128..255]$ is allocated to C.
4. **Free A (Releases $[0..127]$):** Block $[0..127]$ is marked free. Its buddy $[128..255]$ is currently allocated to C, so they cannot merge yet.
5. **Free C (Releases $[128..255]$):** Both buddies $[0..127]$ and $[128..255]$ are now free! They coalesce instantly into a unified $256\ \text{KB}$ block $[0..255]$. This new block's buddy $[256..511]$ is occupied by B, so coalescing halts.

🏛️ Solved Problem: Backpatching Execution Trace for `for` Loop with `break`

Source Statement: `for (i = 0; i < 100; i++) { if (a[i] == 0) break; sum += a[i]; }`

```
100: i = 0                  ; Initialization
101: if i < 100 goto 103    ; Loop condition
102: goto 112               ; falselist: exits for loop
103: t1 = i * 4             ; Array address offset a[i]
104: t2 = a[t1]
105: if t2 == 0 goto 112    ; break statement: jumps directly to exit label 112
106: goto 107
107: t3 = i * 4
108: t4 = a[t3]
109: sum = sum + t4         ; Loop body
110: i = i + 1              ; Step increment
111: goto 101               ; Back-edge to condition check
112:  ...                   ; Target of backpatch(falselist + break_list, 112)
```

Topic 35.5: Object-Oriented Runtime Layout & Virtual Method Tables (Vtables)

In object-oriented languages (such as C++, Java, and C#), member methods may be dynamically overridden in derived subclasses (polymorphism). The compiler realizes **Dynamic Dispatch** via an auxiliary data structure called a **Virtual Method Table (Vtable)**.

Figure 4.2: Object Instance Memory Layout with Virtual Method Table Pointer (`vptr`)

🏛️ Step-by-Step Solved Problem: Dynamic Dispatch Assembly Code Generation

Consider a polymorphic method call `obj->draw()` where `draw()` is at virtual table index 1 (8 bytes offset):

```
MOV RDI, [rbp - 8]     ; Load obj pointer into 1st argument register RDI (the 'this' pointer)
MOV RAX, [RDI]         ; Dereference obj to fetch its Virtual Table Pointer (vptr)
MOV RAX, [RAX + 8]     ; Fetch address of draw() from Vtable slot 1
CALL RAX               ; Indirect procedure call to the concrete method implementation!
```

Complexity: Dynamic dispatch executes in guaranteed $\mathbf{O(1)}$ time with only 2 memory dereferences!

Module V: Code Optimization & Target Code Generation — 12-Topic Master Guide

Topic 36: Issues in the Design of a Code Generator (Target Architecture)

Topic 37: Target Language Instruction Selection & Instruction Cost Models

Topic 38: Addresses in Target Code (Static & Dynamic Offset Calculations)

Topic 39: Basic Blocks Partitioning (The 3 Classical Leader Rules)

Topic 40: Control Flow Graphs (CFG Nodes, Edges, Dominators & Natural Loops)

Topic 41: Next-Use Information & Liveness Analysis Inside Blocks

Topic 42: Register Allocation & Assignment (Chaitin Graph Coloring Algorithm)

Topic 43: Directed Acyclic Graphs (DAG) for Local Basic Block Optimization

Topic 44: Data-Flow Analysis Frameworks (Reaching Definitions & Live Variables)

Topic 45: Loop Optimizations (Loop-Invariant Code Motion & Strength Reduction)

Topic 46: Peephole Optimization Techniques (Sliding Window Transformations)

Topic 47: Simple Code Generator Algorithm (Register & Address Descriptors)

Topic 36 – 38: Fundamental Issues in Target Code Generation

Design Challenge	Engineering Complexity & Tradeoffs	Compiler Strategy
1. Instruction Selection	Mapping uniform Three-Address Code into diverse, irregular target CPU instruction sets (CISC vs. RISC).	Tree-rewriting and dynamic programming instruction selection algorithms.
2. Register Allocation	Registers are the fastest hardware storage ($1\ \text{cycle}$) but strictly limited in quantity (e.g., 16 or 32 physical registers).	NP-Complete **Graph Coloring**; spills excess variables to stack frames.
3. Evaluation Order	The sequence in which computations execute dramatically alters the number of registers required.	Sethi-Ullman algorithm for optimal register labeling of syntax trees.

Topic 39 & 40: Basic Blocks & Control Flow Graphs (CFG)

A **Basic Block** is a maximal sequence of consecutive Three-Address instructions with a **single entry point** (execution enters only at the first instruction) and a **single exit point** (execution leaves only at the last instruction), with zero internal branches or jump targets.

The 3 Classical Rules for Leader Identification

An instruction i is designated a **Leader** if and only if it satisfies at least one condition:

1. **Rule 1:** The very first instruction of the Three-Address Code sequence is a Leader.
2. **Rule 2:** Any instruction that is the **target of a conditional or unconditional goto** is a Leader.
3. **Rule 3:** Any instruction that **immediately follows a conditional or unconditional goto** is a Leader.

Partitioning: A Basic Block starts at a Leader and extends up to, but not including, the next Leader or the end of the program.

🏛️ Step-by-Step Solved Problem: Partitioning TAC into Basic Blocks & Constructing CFG

```
(1)  i = 1             ; LEADER (Rule 1: First instruction)
(2)  j = 1
(3)  t1 = 10 * i       ; LEADER (Rule 2: Target of goto 3 at line 11)
(4)  t2 = t1 + j
(5)  t3 = 8 * t2
(6)  t4 = b[t3]
(7)  a[t3] = t4
(8)  j = j + 1
(9)  if j ⩽ 10 goto 3 ; Branch instruction
(10) i = i + 1         ; LEADER (Rule 3: Immediately follows goto 9)
(11) if i ⩽ 10 goto 3 ; Branch instruction
(12) return            ; LEADER (Rule 3: Immediately follows goto 11)
```

Identified Basic Blocks:

- B_1: Instructions (1) to (2) [Initialization]
- B_2: Instructions (3) to (9) [Inner loop body]
- B_3: Instructions (10) to (11) [Outer loop step]
- B_4: Instruction (12) [Exit]

Control Flow Graph Edges:

$$B_1 \longrightarrow B_2, \quad B_2 \xrightarrow{\text{true}} B_2, \quad B_2 \xrightarrow{\text{false}} B_3, \quad B_3 \xrightarrow{\text{true}} B_2, \quad B_3 \xrightarrow{\text{false}} B_4$$

Topic 42: Register Allocation via Graph Coloring (Chaitin-Briggs)

Kempe's Heuristic for K-Register Graph Coloring: 1. Construct **Interference Graph** $G = (V, E)$ where nodes represent variable live ranges, and edges connect variables simultaneously live at any program point. 2. **Simplify Step:** Find a node v with degree $< K$. Remove v from graph and push onto stack S. 3. If all nodes are removed, pop nodes from S one by one and assign available non-conflicting physical registers. 4. If a stage is reached where all remaining nodes have degree $\geq K$, select a variable to **Spill** to stack memory.

Topic 43: Directed Acyclic Graphs (DAG) for Local Optimization

A **Directed Acyclic Graph (DAG)** constructed for a single basic block exposes common subexpressions, eliminates redundant computations, and detects dead code locally.

🏛️ Step-by-Step DAG Optimization Example

```
// Original Basic Block:
t1 = a + b
t2 = a + b       ; Redundant common subexpression!
t3 = t1 * t2
t4 = c + d
t5 = t3 + t4
```

DAG Construction: Identifies that `t1` and `t2` compute the identical node `+(a, b)`. Replaces `t2` with `t1`.

Optimized Generated Code:

```
t1 = a + b
t3 = t1 * t1      ; Reused t1!
t4 = c + d
t5 = t3 + t4
```

Topic 44 & 45: Principal Global & Loop Optimization Passes

Optimization Pass	Transformation Mechanism	Concrete Code Example
1. Constant Folding	Computes constant operations at compile time rather than generating runtime instructions.	`x = 3.14159 * 2` $\Longrightarrow$ `x = 6.28318`
2. Constant Propagation	Replaces variable uses with known constant values throughout reaching definitions.	`x = 10; y = x + 5;` $\Longrightarrow$ `y = 10 + 5;` $\Longrightarrow$ `y = 15;`
3. Loop-Invariant Code Motion	Hoists statements whose operands do not change during loop iterations into a loop pre-header.	`while (i < N) { x = a + b; ... }` $\Longrightarrow$ `x = a + b; while (i < N) { ... }`
4. Strength Reduction	Replaces expensive operations (multiplication/division) with equivalent cheaper operations (addition/shifts).	`for (i=0; i
5. Dead Code Elimination	Removes statements computing values that are never subsequently read on any execution path.	`if (0) { foo(); }` is entirely deleted from the binary.

Topic 46: Peephole Optimization Techniques

The 4 Classical Peephole Transformations

Operates on a small sliding window (typically 2–4 instructions) of target assembly code:

1. **Eliminating Redundant Load/Store:** `MOV R0, a` immediately followed by `MOV a, R0` (second instruction is deleted).
2. **Eliminating Unreachable Code:** Instructions immediately following an unconditional `JMP` that lack a jump label are deleted.
3. **Algebraic Simplifications:** Replaces `x = x + 0` or `x = x * 1` with no-ops; replaces `x = x * 2` with fast shift `SHL R0, 1`.
4. **Machine Idioms:** Replaces `MOV R0, #0` with single-cycle `XOR R0, R0`.

Topic 47: Simple Code Generation Algorithm

Register Descriptors & Address Descriptors: - **Register Descriptor ($\mathrm{RD}[R]$):** Tracks all variables currently held inside physical register R. - **Address Descriptor ($\mathrm{AD}[v]$):** Tracks all memory and register locations where current value of variable v can be found.

🧠 M5 Active Recall & 10-Question University Exam Master Bank

Q1. Explain the Data-Flow Analysis framework for Reaching Definitions with mathematical equations. (10 Marks)

A definition $d : u = v + w$ **reaches** a point p if there exists an execution path from d to p along which d is not "killed" (redefined).
Data-Flow Transfer Equations:

$$\mathrm{IN}[\mathbf{B}] = \bigcup_{\mathbf{P} \in \mathrm{Pred}[\mathbf{B}]} \mathrm{OUT}[\mathbf{P}]$$

$$\mathrm{OUT}[\mathbf{B}] = \mathrm{GEN}[\mathbf{B}] \cup (\mathrm{IN}[\mathbf{B}] - \mathrm{KILL}[\mathbf{B}])$$

Where $\mathrm{GEN}[B]$ is the set of definitions generated inside block B, and $\mathrm{KILL}[B]$ is the set of all definitions elsewhere in the program that define the same variables modified in B. Solved iteratively to reach maximum fixed-point convergence!

Q2. State the 3 Leader Identification Rules for partitioning Three-Address Code into Basic Blocks. (6 Marks)

1. **Rule 1:** The very first instruction of the Three-Address Code sequence is a Leader.
2. **Rule 2:** Any instruction that is the target of a conditional or unconditional branch is a Leader.
3. **Rule 3:** Any instruction that immediately follows a conditional or unconditional branch is a Leader.

Q3. Explain Loop-Invariant Code Motion and Induction Variable Elimination with concrete code examples. (8 Marks)

• **Loop-Invariant Code Motion (Hoisting):** Computations inside a loop whose operands never change across iterations are moved outside the loop into a newly created pre-header block.
• **Induction Variable Elimination:** When two variables i and j change in lockstep inside a loop (e.g., $j = 4 * i$), j can be updated via addition ($j+ = 4$) and i can be eliminated entirely if it is not used outside the loop.

Q4. What is Peephole Optimization? Explain 4 classical peephole optimization techniques. (8 Marks)

Peephole optimization is a local target-code optimization technique that inspects a short sliding window of target instructions (2 to 4 instructions) to perform immediate peephole replacements:
1. **Redundant Load/Store Elimination:** Deleting redundant `MOV x, R0` immediately after `MOV R0, x`.
2. **Unreachable Code Elimination:** Deleting instructions following an unconditional `JMP` that lack a target label.
3. **Flow of Control Optimization:** Replacing jumps to jumps (`JMP L1` where `L1: JMP L2`) with `JMP L2`.
4. **Algebraic Simplification & Strength Reduction:** Replacing $x = x * 2$ with single-cycle `SHL R0, 1`.

Q5. Explain the Chaitin-Briggs Graph Coloring algorithm for Register Allocation. (10 Marks)

Register allocation is formulated as graph K-coloring where vertices represent variable live ranges and edges represent simultaneous liveness (interference):
1. **Build:** Construct interference graph $G = (V, E)$.
2. **Simplify (Kempe's Heuristic):** Find vertex v with degree $< K$. Remove v and push onto stack S.
3. **Spill:** If all remaining vertices have degree $\geq K$, select a candidate with high spill cost / low frequency to spill to memory.
4. **Select:** Pop vertices from S and assign non-conflicting colors (registers). If spilled, insert load/store instructions and re-run!

Topic 47.2: Comprehensive Data-Flow Analysis & Optimization Traces

🏛️ Solved Problem 1: Reaching Definitions Iterative Fixed-Point Table Convergence

Consider Control Flow Graph with 3 Basic Blocks:

- B_1: $d_1 : i = 1,\ d_2 : j = 1 \implies \text{GEN}[B_1] = \{d_1, d_2\},\ \text{KILL}[B_1] = \{d_3, d_4\}$
- B_2: $d_3 : i = i + 1,\ d_4 : j = j + 1 \implies \text{GEN}[B_2] = \{d_3, d_4\},\ \text{KILL}[B_2] = \{d_1, d_2\}$
- B_3: $d_5 : x = i + j \implies \text{GEN}[B_3] = \{d_5\},\ \text{KILL}[B_3] = \emptyset$

Iterative Fixed-Point Convergence Table:

Iteration	$\text{IN}[B_1]$	$\text{OUT}[B_1]$	$\text{IN}[B_2]$	$\text{OUT}[B_2]$	$\text{IN}[B_3]$	$\text{OUT}[B_3]$
Init	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
Pass 1	$\emptyset$	$\{d_1, d_2\}$	$\{d_1, d_2\}$	$\{d_3, d_4\}$	$\{d_3, d_4\}$	$\{d_3, d_4, d_5\}$
Pass 2	$\emptyset$	$\{d_1, d_2\}$	$\{d_1, d_2, d_3, d_4\}$	$\{d_3, d_4\}$	$\{d_3, d_4\}$	$\{d_3, d_4, d_5\}$
Pass 3 (Converged)	$\emptyset$	$\{d_1, d_2\}$	$\{d_1, d_2, d_3, d_4\}$	$\{d_3, d_4\}$	$\{d_3, d_4\}$	$\{d_3, d_4, d_5\}$

🏛️ Solved Problem 2: Loop-Invariant Code Motion & Induction Variable Strength Reduction

Unoptimized Source Loop:

```
for (i = 0; i < N; i++) {
    x = a + b;           // Loop Invariant!
    arr[i] = i * 4;      // Induction Variable with multiplication!
}
```

Optimized Generated Code:

```
x = a + b;               // Hoisted to loop pre-header
p = &arr[0];             // Pointer initialization
for (i = 0; i < N; i++) {
    *p = i << 2;         // Strength reduction: replaced mult with fast shift & pointer addition
    p++;
}
```

Q6. Compare Local Optimization, Loop Optimization, and Global Optimization across scope and algorithms. (8 Marks)

• **Local Optimization:** Operates strictly within a single Basic Block using Directed Acyclic Graphs (DAG) for common subexpression elimination, algebraic identities, and dead code elimination.
• **Loop Optimization:** Operates on natural loops in the CFG using Loop-Invariant Code Motion (hoisting statements to loop pre-headers), Induction Variable Elimination, and Strength Reduction.
• **Global Optimization:** Operates across multiple basic blocks throughout the entire CFG using Iterative Data-Flow Analysis (Reaching Definitions, Available Expressions, Live Variable Analysis).

Q7. Explain Dominators, Immediate Dominators, and Natural Loops in Control Flow Graphs. (10 Marks)

• **Dominator ($d \ \mathbf{dom}\ n$):** Node d dominates node n if every execution path from the CFG entry node to n must pass through d.
• **Immediate Dominator ($\mathrm{idom}(n)$):** The unique dominator d of n ($d \neq n$) that does not strictly dominate any other strict dominator of n. Forming the **Dominator Tree**.
• **Natural Loop:** A loop characterized by a single loop header d and a **Back-Edge** $n \to d$ where $d \ \mathbf{dom}\ n$. The natural loop consists of d plus all nodes that can reach n without passing through d.

Q8. Explain Available Expressions Data-Flow Analysis and Global Common Subexpression Elimination. (8 Marks)

An expression $x + y$ is **Available** at point p if every path from the initial node to p evaluates $x + y$, and neither x nor y is subsequently redefined.
Transfer Equations (Must-Analysis / Intersection Meet):

$$\mathrm{IN}[\mathbf{B}] = \bigcap_{\mathbf{P} \in \mathrm{Pred}[\mathbf{B}]} \mathrm{OUT}[\mathbf{P}]$$

$$\mathrm{OUT}[\mathbf{B}] = \mathrm{GEN}[\mathbf{B}] \cup (\mathrm{IN}[\mathbf{B}] - \mathrm{KILL}[\mathbf{B}])$$

Used to eliminate redundant evaluations globally across the entire CFG.

Q9. Explain Live Variable Analysis and its role in Dead Code Elimination and Register Allocation. (8 Marks)

A variable v is **Live** at point p if there exists an execution path from p along which v is read before any subsequent redefinition.
Transfer Equations (Backward Data-Flow Analysis):

$$\mathrm{OUT}[\mathbf{B}] = \bigcup_{\mathbf{S} \in \mathrm{Succ}[\mathbf{B}]} \mathrm{IN}[\mathbf{S}]$$

$$\mathrm{IN}[\mathbf{B}] = \mathrm{DEF}[\mathbf{B}] \cup (\mathrm{OUT}[\mathbf{B}] - \mathrm{USE}[\mathbf{B}])$$

Variables not live after a statement are dead (allowing assignment removal), and simultaneously live variables define edges in the Register Interference Graph.

Q10. Explain the Simple Code Generator algorithm with Register Descriptors and Address Descriptors. (10 Marks)

The simple code generator processes Three-Address instructions $x = y \ \mathbf{op}\ z$ sequentially within a basic block:
1. Invokes $\mathbf{getReg}(x = y \ \mathbf{op}\ z)$ to determine the optimal physical register R_y for operand y.
2. If y is not already in R_y, emits `MOV R_y, y`.
3. Emits `OP R_y, z` (where z is in a register or memory location).
4. Updates $\mathrm{RD}[R_y]$ to indicate R_y now contains variable x.
5. Updates $\mathrm{AD}[x]$ to indicate x resides in register R_y, and removes x from other descriptors.

Q11. Explain Constant Folding vs. Constant Propagation with before and after code traces. (6 Marks)

• **Constant Folding:** Evaluates arithmetic operations on constant literals at compile-time: `x = 10 * 20` $\implies$ `x = 200`.
• **Constant Propagation:** Replaces variable uses with their statically known constant values throughout the CFG: `x = 10; y = x + 5;` $\implies$ `y = 10 + 5;` $\implies$ `y = 15;`.

Q12. Explain the Sethi-Ullman Algorithm for Optimal Instruction Scheduling and Register Labeling. (8 Marks)

The **Sethi-Ullman Algorithm** computes the minimum number of registers required to evaluate an expression tree without spilling to memory:
1. Leaves (variables) are assigned label `1`; right leaves of binary operations are assigned `0`.
2. For a binary node with left child label l_1 and right child label l_2:

$$\text{Label}(\text{Node}) = \begin{cases} \max(l_1, l_2) & \text{if } l_1 \neq l_2 \\ l_1 + 1 & \text{if } l_1 = l_2 \end{cases}$$

3. The compiler generates code evaluating the child subtree with the higher label first, minimizing temporary register holding time!

Topic 47.3: Advanced Register Allocation & Global Data-Flow Frameworks

Register allocation is widely considered one of the most critical optimization passes in modern compilers. Because memory access takes 100–300 CPU clock cycles while register access takes only 1 cycle, maximizing variable residence in registers yields order-of-magnitude execution speedups.

🏛️ Complete Step-by-Step Chaitin-Briggs Graph Coloring Trace

Let variables A, B, C, D, E have the following Interference Graph edges for a target CPU with $K = 2$ physical registers (R1, R2):

- A interferes with $\{B, C\}$ (Degree 2)
- B interferes with $\{A, C, D\}$ (Degree 3)
- C interferes with $\{A, B, E\}$ (Degree 3)
- D interferes with $\{B\}$ (Degree 1)
- E interferes with $\{C\}$ (Degree 1)

Graph Coloring Execution Trace:

1. **Step 1:** Node D has degree $1 < 2$. Remove D and push onto stack $S = [D]$.
2. **Step 2:** Node E has degree $1 < 2$. Remove E and push onto stack $S = [D, E]$.
3. **Step 3:** Remaining graph nodes: A (degree 2), B (degree 2), C (degree 2). Since all degrees are ≥ 2, Kempe's heuristic selects A to remove: Stack $S = [D, E, A]$.
4. **Step 4:** Remaining nodes B and C now have degree 1. Remove B: Stack $S = [D, E, A, B]$; Remove C: Stack $S = [D, E, A, B, C]$.
5. **Reconstruction & Color Assignment:**

- Pop C: Assign Register **R1**.
- Pop B: Assign Register **R2** (since B interferes with C).
- Pop A: Interferes with B (R2) and C (R1) $\implies$ **Spill A to Stack Memory!**
- Pop E: Interferes only with C (R1) $\implies$ Assign Register **R2**.
- Pop D: Interferes only with B (R2) $\implies$ Assign Register **R1**.

🏛️ Step-by-Step Dominator Tree & Natural Loop Identification Algorithm

Consider CFG nodes $1 \to 2 \to 3 \to 4 \to 2$ (Back-edge $4 \to 2$):

- Node 1 dominates $\{1, 2, 3, 4\}$
- Node 2 dominates $\{2, 3, 4\}$
- Node 3 dominates $\{3, 4\}$
- Node 4 dominates $\{4\}$

Because $2 \ \mathbf{dom}\ 4$, the edge $4 \to 2$ is a **Back-Edge**. The Natural Loop consists of header node 2 plus all nodes that can reach node 4 without passing through 2, yielding loop node set $\{2, 3, 4\}$.

Topic 47.4: Advanced Instruction Selection & Available Expressions Worklist

Instruction Selection maps the machine-independent Three-Address intermediate code into concrete target processor instructions. The compiler models target CPU instructions as **Tree Patterns (Tiles)** and tiles the intermediate representation Abstract Syntax Tree with minimum cost.

Instruction Selection Technique	Operating Algorithm & Tree Tiling Strategy	Computational Complexity
1. Maximal Munch Algorithm	Greedy top-down pattern matcher: at root of tree, finds the largest matching tile covering the most nodes, emits instruction, and recurses on subtrees.	Linear Time $O(N)$; produces high-quality RISC and CISC instructions.
2. Dynamic Programming Tiling	Bottom-up optimal tiling: computes minimum cost to evaluate each subtree in each hardware register class, then traverses downward to emit instructions.	Optimal Code in $O(N \cdot \|P\|)$ time, where $\|P\|$ is the number of target instruction patterns.
3. Tree-Rewriting / BURG Systems	Uses Tree Grammars and Bottom-Up Rewrite Generators to automatically generate table-driven optimal instruction selectors.	Industry standard in production compilers (GCC, LLVM SelectionDAG).

🏛️ Step-by-Step Solved Problem: Iterative Available Expressions Data-Flow Convergence

Consider a CFG with 3 basic blocks and expression set $U = \{a + b,\ c * d,\ e - f\}$:

- B_1: Computes $a + b$, defines $x \implies \text{GEN}[B_1] = \{a + b\},\ \text{KILL}[B_1] = \emptyset$
- B_2: Computes $c * d$, redefines $a \implies \text{GEN}[B_2] = \{c * d\},\ \text{KILL}[B_2] = \{a + b\}$
- B_3: Computes $e - f \implies \text{GEN}[B_3] = \{e - f\},\ \text{KILL}[B_3] = \emptyset$

Available Expressions Iterative Convergence Table (Must-Analysis: $\text{IN}[B] = \bigcap \text{OUT}[P]$):

Iteration	$\text{IN}[B_1]$	$\text{OUT}[B_1]$	$\text{IN}[B_2]$	$\text{OUT}[B_2]$	$\text{IN}[B_3]$	$\text{OUT}[B_3]$
Init	$\emptyset$	U	U	U	U	U
Pass 1	$\emptyset$	$\{a + b\}$	$\{a + b\}$	$\{c * d\}$	$\{c * d\}$	$\{c * d, e - f\}$
Pass 2 (Converged)	$\emptyset$	$\{a + b\}$	$\{a + b\}$	$\{c * d\}$	$\{c * d\}$	$\{c * d, e - f\}$

🏛️ Solved Problem: DAG Construction for 8-Statement Basic Block

Basic Block Statements:

```
(1) a = b + c
(2) d = b + c       ; Common Subexpression: Reuses DAG node +(b, c)
(3) e = a + d
(4) f = b + c       ; Common Subexpression: Reuses DAG node +(b, c)
(5) g = e * f
(6) h = a + d       ; Common Subexpression: Reuses DAG node +(a, d)
(7) i = g - h
(8) x = i
```

Optimized Target Code Generated from DAG:

```
a = b + c
d = a               ; Copy instead of addition
e = a + a
g = e * a
h = e               ; Copy instead of addition
x = g - e           ; Direct assignment eliminating temporaries f, h, i
```

Topic 47.5: Interprocedural Optimization (IPO) & Whole-Program Compilation

While intraprocedural optimization operates within a single procedure, **Interprocedural Optimization (IPO)** analyzes relationships across the entire call graph of the program.

Interprocedural Pass	Operating Mechanism	Optimization Impact & Tradeoffs
1. Function Inlining	Replaces a procedure call site with the verbatim body of the callee, substituting actual parameters for formal variables.	Eliminates function call overhead (10 to 30 cycles) and enables massive downstream constant propagation; may increase code size.
2. Interprocedural Constant Propagation (IPCP)	Tracks constant values passed as arguments across procedure boundaries throughout the Call Graph.	Specializes procedures for common constant inputs and prunes dead branches globally.
3. Devirtualization	Proves via Class Hierarchy Analysis (CHA) that a virtual call site has only one possible target subclass.	Converts expensive indirect virtual dispatch calls into direct static calls or inlined code!

🏛️ Step-by-Step Solved Problem: Function Inlining & Downstream Optimization Cascade

Original Code:

```
int square(int val) { return val * val; }
int compute() {
    int x = 5;
    return square(x) + 10;
}
```

Optimization Cascade Trace:

1. **Pass 1 (Function Inlining):** Replaces `square(x)` with `(x * x)` $\implies$ `return (x * x) + 10;`
2. **Pass 2 (Constant Propagation):** Replaces `x` with `5` $\implies$ `return (5 * 5) + 10;`
3. **Pass 3 (Constant Folding):** Evaluates `5 * 5 = 25` and `25 + 10 = 35` at compile time.
4. **Final Generated Target Code:** `MOV EAX, 35; RET;` (Zero memory access, zero multiplication, instant return!)